

UNIVERSITÀ DEGLI STUDI ROMA TRE

Dipartimento di Ingegneria Civile, Informatica
e delle Tecnologie Aeronautiche
Corso di Laurea Ingegneria Informatica e dell'Intelligenza Artificiale

Tesi di Laurea

Agent-based Earth Observation discovery: un approccio multi-agentico per l'accesso al dato nei cataloghi di osservazione della Terra

Laureando

Alessio Pecilli

Matricola 589232

Relatore

Prof. Matteo Rosati

Anno Accademico 2025/2026

Sommario

La consultazione delle fonti di osservazione della Terra richiede di distinguere eventi naturali, acquisizioni satellitari, attivazioni cartografiche e cataloghi di dataset. Questa tesi presenta Copernicus Data Workspace, un prototipo conversazionale che interpreta richieste geografiche e coordina il recupero di informazioni mantenendone riconoscibili provenienza e limiti.

Il sistema adotta un workflow LangGraph con componenti decisionali basati su modelli linguistici e trasformazioni deterministiche. Il percorso di Discovery interroga un catalogo di metadati NebulaGraph alimentato da harvester Marine, STAC e Drought. Il percorso operativo consulta GDACS, Copernicus Emergency Management Service e il catalogo STAC remoto; MCP delimita le integrazioni GDACS e CEMS. Memoria conversazionale, artefatti e telemetria sostengono rispettivamente i turni successivi, la presentazione dei risultati e la diagnosi. Il modello interpreta e spiega, mentre conteggi e aggregazioni delle visualizzazioni derivano dai record recuperati.

La valutazione distingue controlli software, benchmark del workflow e verifica dell'interfaccia. Il protocollo descritto combina controlli deterministici e LLM-as-a-Judge, rendendo espliciti denominatori, dipendenze tra casi e limiti del valutatore. Sul catalogo a grafo il sistema supera circa due casi su tre, con il limite principale nell'ordinamento dei risultati; sulle acquisizioni satellitari supera tutti i casi del benchmark. Il contributo è progettuale e implementativo. I passi successivi sono un confronto controllato con un agente singolo o con una pipeline più semplice, la validazione umana dei giudizi e prove conversazionali nel browser.

Indice

Sommario **ii**

Indice **iii**

1 Introduzione **1**

1.1 Posizionamento rispetto ai lavori correlati 2

1.2 Il problema affrontato 2

1.3 Obiettivo del lavoro 3

1.4 Impostazione del progetto 5

1.5 Perimetro e limiti 6

2 Dall'intelligenza artificiale agli agenti basati su LLM **7**

2.1 Intelligenza artificiale e nozione di agente 7

2.2 Apprendimento statistico e reti profonde 8

2.3 Modelli linguistici e architettura Transformer 10

2.4 Agenti basati su LLM 12

3 Model Context Protocol: contesto, strumenti e interoperabilità **14**

3.1 Problema e scelta architetturale 14

3.2 Concetti principali 15

3.3 Capacità esposte dai server 16

3.4 Uso di MCP nel workflow 17

3.5 Confronto e limiti 17

3.6 Sicurezza e gestione operativa 18

3.7 Criteri adottati nel progetto 19

4 Fonti informative geospaziali: GDACS e Copernicus 20

4.1 GDACS 20
4.2 Il programma Copernicus 22
4.3 Copernicus Data Space Ecosystem e STAC 23
4.4 Copernicus Emergency Management Service 24
4.5 Marine e Drought nel catalogo di Discovery 24
4.6 Confronto e complementarità 26
4.7 Implicazioni per la progettazione dell'agente 26

5 Architettura del sistema proposto 27

5.1 Obiettivo e requisiti 27
5.2 Componenti applicativi 28
5.3 Vista architetturale 31
5.4 Organizzazione del backend e del frontend 32
5.5 Il grafo degli agenti 33
5.6 Catalogo a grafo e pipeline di harvesting 36
5.7 Agenti specializzati e nodi di elaborazione 38
5.8 Contratti e stato condiviso 38
5.9 Persistenza 39
5.10 Dashboard generativa e verificabile 40
5.11 Integrazione MCP 41
5.12 Distribuzione tramite container 41
5.13 Sicurezza e osservabilità 42

6 Funzionamento end-to-end, casi d'uso e verifica 44

6.1 Ingresso della richiesta 44
6.2 Scenario 1: analisi multi-sorgente di un pericolo 45
6.3 Scenario 2: interrogazione di una attivazione CEMS 46
6.4 Scenario 3: follow-up e riuso della memoria 47
6.5 Scenario 4: scoprire un dataset nel grafo 49

6.6 Dashboard Copernicus 49
6.7 Streaming della risposta 52
6.8 Gestione degli errori 52
6.9 Matrice dei percorsi e delle evidenze 53
6.10 Strategia di verifica 55
6.11 Limiti attuali 56

7 Osservabilità del workflow e diagnosi 57
7.1 Log, metriche e trace 57
7.2 Architettura della telemetria 58
7.3 Che cosa viene misurato 58
7.4 Come leggere le metriche 60
7.5 Dashboard Grafana e alert 60
7.6 Diagnosi e copertura 61

8 Valutazione con LLM-as-a-Judge 64
8.1 Oggetto e perimetro della valutazione 64
8.2 Dataset ed evidenze di riferimento 65
8.3 Esecuzione del vero workflow 66
8.4 Controlli deterministici e rubriche ordinali 66
8.5 Stati finali e denominatori 68
8.6 Incertezza e misure di recupero 69
8.7 Confronti e ripetizioni 70
8.8 Risultati 70
8.9 Interpretazione e minacce alla validità 72

9 Conclusioni e sviluppi futuri 73

Bibliografia 75

Capitolo 1

Introduzione

Una domanda relativa a un'emergenza naturale raramente trova risposta in una sola fonte. Sapere se si è verificato un terremoto, verificare quali acquisizioni satellitari coprano la zona e controllare se sia stata attivata una produzione cartografica sono operazioni diverse. I dati possono riguardare lo stesso luogo e lo stesso intervallo temporale, ma descrivono fenomeni distinti e hanno tempi di aggiornamento, formati e livelli di affidabilità propri.

Si consideri, ad esempio, una richiesta sui terremoti avvenuti in Italia nell'ultima settimana e sulla relativa copertura satellitare. La prima parte richiede un elenco di eventi e dei rispettivi livelli di allerta; la seconda riguarda la disponibilità di osservazioni della Terra. Un'eventuale attivazione di un servizio di emergenza aggiunge poi informazioni su aree di interesse e prodotti cartografici. Riunire questi elementi in una sola risposta è utile soltanto se la loro provenienza rimane riconoscibile.

Questa tesi affronta il problema realizzando *Copernicus Data Workspace*: uno spazio di lavoro conversazionale che coordina fonti geospaziali indipendenti. L'utente formula la richiesta in linguaggio naturale; il sistema ne ricava luogo, periodo e obiettivo, sceglie le fonti pertinenti e restituisce testo, dati strutturati e visualizzazioni. Il modello linguistico fa da interprete: comprende la domanda e formula la risposta, mentre gli eventi e i valori mostrati nei grafici provengono dalle fonti consultate.

1.1 Posizionamento rispetto ai lavori correlati

Il progetto si colloca nella linea degli agenti basati su LLM che usano strumenti esterni. ReAct studia l'alternanza tra ragionamento e azione; Toolformer affronta l'apprendimento dell'uso delle API [40, 34]. Il function calling documenta invece un meccanismo applicativo per collegare il modello a funzioni eseguibili [26].

Nel dominio dell'osservazione della Terra, Earth Copilot nasce dalla collaborazione NASA–Microsoft per rendere interrogabili dati geospaziali attraverso domande in linguaggio naturale [20]; la documentazione pubblica ne descrive finalità e integrazione con l'accesso ai dati NASA. Il CDSE STAC Browser offre invece navigazione e filtri sui metadati del catalogo, con accesso alle proprietà e agli asset dei prodotti [5]. È un termine di confronto per la consultazione dei cataloghi, mentre il workspace qui proposto aggiunge interpretazione linguistica, coordinamento di fonti eterogenee e continuità conversazionale.

Il posizionamento del lavoro riguarda l'integrazione di questi bisogni: Discovery di risorse catalogate, ricerca STAC live e consultazione di eventi e attivazioni. MCP [21] delimita l'accesso a GDACS e CEMS; il catalogo a grafo conserva le gerarchie delle risorse e il ranking applica regole esplicite. Il contributo è un'architettura applicativa, costruita su tecniche di retrieval esistenti, insieme al relativo protocollo di verifica.

1.2 Il problema affrontato

Le applicazioni conversazionali rendono semplice l'accesso a servizi che, presi singolarmente, richiederebbero la conoscenza di endpoint, parametri e formati di risposta. La complessità dei dati sottostanti, però, rimane. Nel percorso operativo sono coinvolte tre famiglie di informazioni, alle quali si affianca un catalogo trasversale di metadati.

GDACS pubblica eventi e allerte relativi a disastri naturali. Il catalogo del Copernicus Data Space Ecosystem descrive invece le acquisizioni satellitari disponibili per una certa area e un certo periodo. Copernicus Emergency Management Service documenta attivazioni e prodotti cartografici predisposti a supporto della gestione delle emergenze. Queste fonti possono essere consultate nello stesso turno e ciascuna conserva il proprio significato: il numero di acquisizioni satellitari, per esempio, misura la disponibilità

di osservazioni, mentre una prima stima della gravità di un evento viene dai livelli di allerta GDACS.

Alla distinzione semantica si aggiunge quella tecnica. GDACS distribuisce anche feed XML, il catalogo satellitare espone un'interfaccia STAC e CEMS adotta strutture specifiche per le proprie attivazioni. I servizi possono restituire risposte vuote, diventare temporaneamente indisponibili o richiedere trattamenti diversi degli errori. Collocare questi dettagli nei prompt renderebbe difficile capire se un'informazione sia stata recuperata oppure generata dal modello.

Anche l'interpretazione linguistica deve essere verificata. Un LLM sa trattare espressioni come "la settimana scorsa", riconoscere un codice CEMS e comporre un testo leggibile, ma la sua conoscenza può essere incompleta o datata: una frase plausibile va riscontrata nella fonte consultata. Nel progetto interpretazione, accesso ai servizi e calcoli sono perciò passaggi separati.

I turni successivi richiedono inoltre di conservare il contesto. Dopo una tabella o un grafico, l'utente può chiedere di mostrare una sola piattaforma, cambiare l'aggregazione o aggiornare una sorgente. Il significato di questi comandi dipende dai risultati precedenti; lo stato è quindi associato alla conversazione, mentre i dati remoti restano fuori dalla cronologia testuale.

La verificabilità riguarda sia il testo sia le visualizzazioni. Link, identificativi, timestamp e criteri di aggregazione fanno parte del risultato. Un grafico deve indicare i record e i campi dai quali deriva; il modello può proporre la vista, mentre conteggi e misure vengono calcolati dal software.

1.3 Obiettivo del lavoro

L'obiettivo è un prototipo che trasformi una richiesta geografica in un percorso di consultazione controllato. Il sistema deve poter interrogare più fonti quando la domanda lo richiede, continuare a produrre una risposta anche quando una sorgente secondaria è indisponibile e mantenere il contesto necessario per i turni successivi. I risultati devono restare associati alla fonte dalla quale provengono.

La tesi impiega modelli linguistici esistenti e studia l'architettura che li circonda: ammissione della domanda nel dominio, risoluzione dei parametri, scelta delle sorgenti

e restituzione dei risultati nel workspace. Le ambiguità linguistiche sono affidate al modello; schemi, filtri, calcoli e persistenza rimangono sotto il controllo del codice.

Il contributo dichiarato è **progettuale e implementativo**: un prototipo con responsabilità specializzate, percorsi Discovery/operativo controllati e artefatti verificabili. Il titolo parla di approccio multiagentico perché il workflow coordina componenti decisionali basati su LLM con responsabilità specializzate, affiancati da nodi deterministici di recupero e trasformazione.

Il percorso **operativo** combina la ricerca web con tre fonti strutturate. GDACS fornisce eventi e allerte; il catalogo STAC di Copernicus descrive acquisizioni satellitari e relativa copertura; CEMS permette di consultare attivazioni di Rapid Mapping e Risk and Recovery Mapping. Il contenuto della domanda determina quali percorsi siano necessari e più rami possono essere attivati nello stesso turno.

Il percorso di **Discovery** affronta una domanda complementare: quali risorse Copernicus permettono di studiare una certa variabile, area o periodo? Tre harvester raccolgono metadati Marine, CDSE/STAC e Drought in NebulaGraph. Il workflow consulta questo grafo per individuare dataset, variabili, collezioni e layer pertinenti. La Discovery indica quali risorse esistono, per esempio un dataset di temperatura, e come raggiungerle; la ricerca delle singole acquisizioni resta affidata al servizio STAC remoto. Sono ammessi anche percorsi **combinati**, in cui Discovery e analisi operativa convivono nella stessa sessione mantenendo distinti metadati di catalogo e scene satellitari live.

Per GDACS e CEMS è stato adottato il Model Context Protocol. I dettagli dei due servizi vengono incapsulati in server separati, ai quali il backend accede attraverso un client comune. Il catalogo STAC live rimane integrato direttamente nell'applicazione. La presenza dei due approcci nello stesso progetto permette di confrontarne gli effetti sull'organizzazione del codice.

Il punto di ingresso è una dashboard web autenticata con Keycloak: home in italiano con selettore IT/EN, formulazione della domanda, suggerimenti e ripresa delle indagini recenti. La risposta operativa può includere una dashboard delle acquisizioni Copernicus. Grafici, indicatori, tabella e mappa derivano dai record STAC normalizzati; GDACS, CEMS e ricerca web restano nella parte testuale o in sezioni dedicate, perché le rispettive quantità hanno natura diversa. Durante la conversazione l'utente può applicare filtri,

cambiare vista o chiedere una nuova aggregazione. Ogni modifica produce una revisione dell'artefatto e i valori sono sempre ricalcolati sul dataset conservato.

La Discovery restituisce invece un artefatto di risorse catalogate con identificativi, descrizioni e collegamenti. Sono quindi distinte la vista dei metadati e quella delle acquisizioni. A entrambe si affiancano un'infrastruttura di osservabilità, per ricostruire tempi ed errori, e un protocollo di valutazione del workflow, descritto separatamente nel Capitolo 8.

1.4 Impostazione del progetto

L'intera piattaforma (backend, frontend, server MCP, Redis, Keycloak e harvester NebulaGraph) viene avviata come un insieme coordinato di container. La domanda viene prima confrontata con il dominio geografico e ambientale, poi trasformata in una rappresentazione strutturata e assegnata alle sorgenti pertinenti. Un grafo LangGraph stabilisce i percorsi consentiti e permette ai rami indipendenti di procedere in parallelo. La struttura rende ispezionabili le condizioni applicate prima di raccogliere i risultati.

Ogni integrazione traduce la risposta remota in un dataset con campi stabili. I client possono così essere provati con input controllati e gli errori restano associati al servizio che li ha prodotti. Se un ramo fallisce, i risultati già disponibili vengono conservati e la risposta segnala la fonte mancante.

La memoria è gestita come stato applicativo. A ogni conversazione è associato un checkpoint Redis che comprende i messaggi necessari, i risultati delle sorgenti e i riferimenti agli artefatti. Un turno successivo può così riutilizzare un dataset o aggiornare soltanto la parte indicata dall'utente. La memoria è propria di ciascuna conversazione e lascia invariati i parametri del modello.

La verifica software copre parsing dei feed, filtri e aggregazioni con test unitari; i percorsi del workflow vengono provati con dipendenze simulate; API, persistenza e operazioni sulla dashboard sono controllate dai test di integrazione. La valutazione end-to-end del comportamento dell'assistente è affidata al protocollo 6.0 (dataset corrente da 105 domande, controlli deterministici e LLM-as-a-Judge), descritto nel Capitolo 8. Dopo le fonti e l'architettura, il Capitolo 6 presenta i percorsi applicativi; i Capitoli 7 e 8 approfondiscono rispettivamente osservabilità e valutazione.

1.5 Perimetro e limiti

La descrizione dell'implementazione si basa sul codice del prototipo e sulla guida tecnica del progetto [29].

Il sistema è un prototipo di ricerca: le comunicazioni ufficiali in caso di emergenza restano di competenza delle autorità di protezione civile. GDACS offre valutazioni rapide che possono cambiare quando vengono raccolte nuove informazioni. Le attivazioni CEMS documentano prodotti richiesti e realizzati in uno specifico contesto operativo. Il catalogo STAC indica quali osservazioni sono disponibili; la stima dell'entità di un danno richiede analisi specifiche.

La dashboard descrive la disponibilità delle acquisizioni. La heatmap rappresenta la densità dei centri delle bounding box degli item recuperati e i rettangoli visualizzati mostrano le estensioni delle bounding box, che approssimano le geometrie degli item. L'analisi scientifica delle immagini (indici spettrali, classificazioni del suolo, stime degli effetti di un evento) esula dal prototipo.

La copertura geografica dipende dai servizi consultati. Un luogo espresso in linguaggio naturale deve essere risolto in coordinate o in una bounding box; nomi ambigui possono richiedere un chiarimento. Ricerca web, cataloghi e feed dipendono inoltre dalla rete e dalla disponibilità dei rispettivi fornitori: il sistema gestisce errori e risultati parziali e segnala le fonti irraggiungibili.

Sul piano della valutazione, l'organizzazione multiagente è descritta e verificata, ma manca ancora un confronto controllato con un agente singolo o con una pipeline più semplice nelle stesse condizioni; un vantaggio quantitativo di accuratezza, latenza o costo resta quindi da misurare. I giudizi dell'LLM-as-a-Judge attendono una validazione umana e la verifica end-to-end delle conversazioni nel browser è ancora da svolgere. Sul piano della sicurezza, Keycloak protegge l'interfaccia, mentre le API del backend devono ancora verificare il token e il proprietario degli artefatti.

Per questo le risposte conservano collegamenti e identificativi, distinguono i dati recuperati dal testo generato e segnalano le sorgenti mancanti. Il criterio adottato è rendere riconoscibile il percorso con cui il risultato è stato costruito, anche quando alcune fonti sono assenti.

Capitolo 2

Dall'intelligenza artificiale agli agenti basati su LLM

La progettazione dell'assistente descritto in questa tesi richiede di distinguere il modello linguistico dal sistema che lo utilizza. Un LLM genera testo a partire dal contesto ricevuto; nel prototipo il ciclo agentivo aggiunge gestione dello stato, scelta degli strumenti e osservazione degli esiti. Il capitolo concentra la discussione sull'apprendimento statistico, sull'architettura Transformer e sui limiti degli LLM isolati, per poi tornare alla nozione di agente adottata nel progetto.

2.1 Intelligenza artificiale e nozione di agente

Russell e Norvig distinguono le definizioni dell'intelligenza artificiale in base al riferimento al pensiero, al comportamento umano o alla razionalità. Per descrivere il sistema sviluppato è sufficiente la loro prospettiva agentiva: un agente riceve percezioni dall'ambiente e sceglie azioni [33]. Questa definizione permette di discutere il comportamento del software senza attribuire al modello proprietà cognitive: l'analisi resta sul piano del comportamento osservabile.

Formalmente, un agente può essere rappresentato come una funzione che trasforma la storia delle osservazioni in un'azione. Se $p_1, p_2, \ldots, p_t$ sono le percezioni disponibili

fino al tempo t, l'agente calcola una politica:

$$\pi(p_1, p_2, \ldots, p_t) = a_t$$

dove a_t è l'azione scelta. Nell'assistente oggetto della tesi, la storia comprende i messaggi e i risultati restituiti dalle fonti; le azioni sono il routing verso un ramo del workflow, l'invocazione di un tool o la produzione della risposta.

Un classificatore può essere un componente della funzione di un agente; il ciclo applicativo di questo progetto comprende però anche stato, strumenti ed esiti delle azioni. Nel seguito il termine agente basato su LLM indica un componente che usa il modello per scegliere un'azione nel workflow; repository, filtri e ranking deterministici sono invece nodi di elaborazione.

2.2 Apprendimento statistico e reti profonde

2.2.1 Dai sistemi simbolici all'apprendimento dai dati

La ricerca degli anni Cinquanta adottò soprattutto rappresentazioni simboliche: fatti, predicati e regole di inferenza venivano specificati esplicitamente. Da questa impostazione derivano, tra l'altro, la pianificazione classica, la dimostrazione automatica di teoremi e i sistemi esperti. Il metodo resta efficace nei domini formalizzabili, ma richiede un aggiornamento manuale quando cambiano regole e conoscenza del dominio.

Il machine learning affronta diversamente il problema: il programmatore definisce una famiglia di modelli e una procedura di apprendimento, mentre i parametri sono stimati a partire dagli esempi. Sistemi simbolici e modelli appresi possono comunque convivere: nell'architettura descritta più avanti, le decisioni probabilistiche dell'LLM convivono con regole e schemi di validazione scritti nel codice.

2.2.2 Machine learning

Nel machine learning una funzione sconosciuta viene approssimata a partire dai dati osservati. Dato un insieme di esempi:

$$D = \{(x_i, y_i)\}_{i=1}^{n}$$

si cerca una funzione f_θ parametrizzata da θ tale che $f_\theta(x_i)$ sia vicino a y_i. In classificazione, x_i può essere un'immagine e y_i una classe. In regressione, y_i può essere un valore numerico. In apprendimento non supervisionato, l'obiettivo è scoprire struttura nei dati: cluster, rappresentazioni compatte, anomalie o fattori latenti.

Rispetto a un programma tradizionale, parte del comportamento deriva dai dati anziché dal codice. Il programmatore definisce architettura, procedura di addestramento e funzione obiettivo; il risultato dipende anche dai dati e dai parametri appresi.

Un modello supervisionato viene allenato minimizzando una loss:

$$\theta^* = \arg\min_\theta \frac{1}{n} \sum_{i=1}^{n} L(f_\theta(x_i), y_i)$$

dove L misura l'errore tra predizione e valore atteso. L'architettura di f_θ, i dati e la funzione di perdita dipendono dal problema, ma i parametri vengono in ogni caso aggiornati in funzione degli esempi osservati.

Il machine learning ha reso possibile trattare problemi in cui scrivere regole esplicite era troppo costoso o fragile. Tuttavia, molti modelli classici richiedevano ancora una fase di feature engineering: un esperto doveva decidere quali caratteristiche estrarre dai dati grezzi. In visione artificiale, per esempio, prima delle reti profonde si progettavano descrittori di bordi, texture o forme. In elaborazione del linguaggio naturale si usavano n-grammi, dizionari, feature sintattiche e rappresentazioni sparse.

2.2.3 Deep learning

Il deep learning estende il machine learning usando reti neurali con molti strati, capaci di apprendere rappresentazioni gerarchiche. Ogni strato trasforma l'input in una rappresentazione intermedia sulla quale opera lo strato successivo. In una rete per immagini, per esempio, i primi livelli possono rilevare pattern locali e quelli finali concetti più vicini alle classi di interesse.

LeCun, Bengio e Hinton descrivono il deep learning come una famiglia di metodi in cui modelli composti da più livelli apprendono rappresentazioni con diversi livelli di astrazione [18]. Le trasformazioni utili al compito vengono apprese dai dati, riducendo il ricorso a feature progettate manualmente.

La disponibilità di dataset estesi e GPU, insieme ai progressi nelle architetture e nelle procedure di ottimizzazione, ha reso praticabile l'addestramento di reti sempre più profonde.

Nel 2012 AlexNet ottenne su ImageNet un errore di classificazione sensibilmente inferiore rispetto agli altri sistemi presentati nella stessa competizione [14]. Il risultato contribuì alla diffusione delle reti profonde nella visione artificiale e mostrò l'effetto congiunto di dataset estesi, GPU e architetture adatte al compito.

Una parte della complessità si sposta così dalle condizioni scritte nel codice ai parametri: il comportamento dipende congiuntamente da dati, architettura e procedura di addestramento.

2.3 Modelli linguistici e architettura Transformer

2.3.1 Modellazione del linguaggio

Il linguaggio naturale presenta ambiguità, dipendenze dal contesto e riferimenti a conoscenze implicite. I sistemi di NLP hanno a lungo combinato regole linguistiche, feature manuali e modelli statistici; il deep learning ha poi favorito rappresentazioni dense e reti neurali sequenziali.

Un modello linguistico assegna probabilità a sequenze di token. Se una frase viene rappresentata come $x_1, x_2, \ldots, x_n$, un modello autoregressivo stima:

$$P(x_1, \ldots, x_n) = \prod_{t=1}^{n} P(x_t \mid x_1, \ldots, x_{t-1})$$

Nel pre-addestramento autoregressivo, la funzione obiettivo favorisce la predizione del token successivo dato il contesto precedente. L'apprendimento può così catturare regolarità grammaticali, semantiche e fattuali dei dati, senza garantire che ogni relazione appresa sia vera o che ogni fatto venga conservato correttamente. Altri obiettivi intervengono nei successivi adattamenti.

Il tokenizer converte il testo in unità discrete, che possono coincidere con una parola, una sua parte o un segno di punteggiatura. A ciascuna unità è associato un identificativo numerico, trasformato poi in un embedding sul quale opera il modello.

2.3.2 Transformer e attenzione

L'architettura di riferimento per gli LLM è il Transformer, introdotto da Vaswani et al. in *Attention Is All You Need* [39]. I precedenti modelli sequenziali usavano spesso reti ricorrenti o convoluzionali. Le reti ricorrenti elaborano la sequenza un elemento alla volta e mantengono uno stato nascosto, con minori possibilità di parallelizzare l'addestramento e difficoltà nel rappresentare dipendenze molto lunghe.

Il Transformer usa invece il meccanismo di self-attention. Ogni token costruisce una rappresentazione guardando gli altri token del contesto e pesandoli in base alla loro rilevanza. In forma semplificata, l'attenzione usa tre matrici: query Q, key K e value V. Il punteggio tra un token e un altro viene calcolato confrontando query e key; se d_k è la dimensione delle key per testa, i value vengono combinati con pesi normalizzati:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Il calcolo mette in relazione direttamente posizioni diverse della sequenza, con una maschera causale che, nel caso autoregressivo, esclude i token futuri, senza comprimere tutto il prefisso in un singolo stato ricorrente. Le diverse teste di attenzione apprendono relazioni differenti in parallelo. Questa struttura facilita la parallelizzazione dell'addestramento e si è dimostrata adatta a scalare con dati e modelli di grandi dimensioni.

2.3.3 Modelli linguistici di grandi dimensioni

Con LLM si indica un modello linguistico neurale addestrato su collezioni testuali estese, in genere basato su Transformer. Il termine "grande" può riferirsi al numero di parametri e alla scala dei dati e del calcolo impiegati; la lunghezza del contesto è una proprietà distinta.

I parametri sono i valori numerici appresi, organizzati principalmente in matrici di pesi e vettori di bias. GPT-3, per esempio, è stato presentato come un modello autoregressivo da 175 miliardi di parametri [3]. La qualità di un modello dipende, oltre che dalla dimensione, da architettura, dati e addestramento, mentre l'aumento dei parametri comporta maggiori costi di memoria e calcolo.

Un parametro è un coefficiente che contribuisce a molte predizioni. L'informazione appresa è distribuita tra i pesi, mentre un database conserva record espliciti e li recupera senza generarli. La distinzione è rilevante per questo lavoro: i dati su eventi e acquisizioni satellitari devono provenire dalle fonti interrogate dal sistema anziché dalla memoria parametrica del modello.

Durante l'inferenza, il modello riceve un prompt, cioè una sequenza di input, e produce una distribuzione di probabilità sul token successivo. La generazione procede iterativamente: il token scelto viene aggiunto al contesto e il processo continua. Parametri di decoding come temperatura, top-p o penalità di ripetizione modificano il modo in cui si campiona dalla distribuzione, influenzando determinismo e varietà dell'output.

2.3.4 Pre-addestramento, adattamento e limiti degli LLM isolati

Il ciclo tipico di un LLM moderno comprende il pre-addestramento su grandi collezioni di testo e, spesso, un successivo adattamento alle istruzioni. Nel contesto applicativo di questa tesi, l'uso del solo modello lascia irrisolti tre problemi:

- **Conoscenza statica**: i parametri riflettono il training set e restano fermi alla data dell'addestramento;
- **Allucinazione**: il modello può generare testo plausibile ma falso;
- **Assenza di effetti esterni**: un modello che produce solo testo ha bisogno di componenti esterni per eseguire comandi, leggere documenti aggiornati o interagire con il mondo.

Fonti aggiornate, memoria e azioni devono quindi essere fornite da componenti esterni al modello.

2.4 Agenti basati su LLM

Nelle architetture considerate in questa tesi, un agente IA è un sistema software che usa un LLM per scegliere e coordinare azioni. Attorno al modello è implementato un ciclo di controllo che riceve un obiettivo, osserva il contesto, invoca strumenti esterni e aggiorna lo stato in base ai risultati ottenuti.

La letteratura recente ha studiato esplicitamente l'alternanza tra ragionamento e azione. ReAct combina tracce di ragionamento e invocazioni di strumenti nello stesso processo decisionale [40]; Toolformer affronta invece il problema di apprendere quando e come richiamare API esterne [34]. Il progetto qui presentato applica questi principi, con modelli già addestrati, a un workflow LangGraph controllato (*Copernicus Data Workspace*), nel quale le decisioni dell'LLM sono validate da schemi e regole prima dell'esecuzione sulle sorgenti.

Il passaggio dai sistemi simbolici agli agenti contemporanei è graduale: regole esplicite e modelli appresi convivono nelle stesse architetture. In questa prospettiva, il modello linguistico è una componente dell'agente, affiancata da altre. Nel prototipo azioni, memoria Redis, artefatti dashboard e controlli restano affidati a componenti software separati. Il capitolo seguente esamina MCP, il protocollo adottato per collegare alcuni di questi componenti a GDACS e CEMS.

Capitolo 3

Model Context Protocol: contesto, strumenti e interoperabilità

3.1 Problema e scelta architetturale

L'assistente sviluppato per questa tesi consulta servizi con interfacce e formati diversi. GDACS pubblica feed RSS in XML, il catalogo CDSE usa richieste STAC e Copernicus EMS restituisce strutture specifiche del servizio. Gestire questi dettagli direttamente nei nodi del grafo avrebbe legato l'orchestrazione e i prompt a trasporto, autenticazione, parsing e messaggi di errore delle singole sorgenti.

Il Model Context Protocol (MCP) è stato adottato per i rami GDACS e Copernicus EMS allo scopo di introdurre un confine tra il workflow e l'accesso ai dati. La specifica definisce un protocollo aperto per collegare applicazioni basate su LLM a strumenti e risorse esterne [21]; la proposta originaria insiste in particolare sulla riusabilità delle integrazioni tra host differenti [1].

Il capitolo si concentra sul percorso effettivamente usato nel progetto: host, client, server e invocazione dei tool.

MCP propone un modello client-server. L'applicazione IA agisce come host: contiene o coordina il modello, presenta l'interfaccia utente e gestisce le policy. Al suo interno

usa uno o più client MCP per connettersi a server MCP. Ogni server espone capacità specifiche: strumenti, risorse o prompt. L'agente può scoprire queste capacità e usarle attraverso messaggi standardizzati.

3.2 Concetti principali

3.2.1 Host

L'host coordina modello, client e policy applicative. Nel caso studiato questo ruolo appartiene al backend FastAPI e al workflow LangGraph; la dashboard nativa fornisce l'interfaccia con l'utente (Open WebUI resta un client legacy opzionale).

È l'host a scegliere quali capacità mostrare al modello. Nel progetto la scelta è ristretta a operazioni di consultazione: i server MCP offrono soltanto letture e ricerche.

3.2.2 Client

Il client MCP è il componente che mantiene la connessione con un server. In molte architetture è una parte interna dell'host, invisibile all'utente. Il client si occupa di inizializzare la sessione, negoziare capacità, inviare richieste e ricevere risposte.

Se l'host usa più server, mantiene più connessioni. Il backend sviluppato in questa tesi utilizza un client comune e apre una sessione distinta per GDACS e Copernicus EMS.

3.2.3 Server

Il server MCP espone funzionalità a cui l'agente può accedere. Può essere locale, per esempio un processo avviato sulla macchina dell'utente, oppure remoto, raggiungibile via rete. Un server può offrire:

- **tools**, cioè azioni invocabili;
- **resources**, cioè dati leggibili e indirizzabili;
- **prompts**, cioè template o procedure testuali riusabili.

La distinzione separa dati, azioni e istruzioni riutilizzabili. Nel progetto i server GDACS e CEMS usano soltanto la prima categoria: espongono come tool le azioni di lettura e ricerca.

3.2.4 Trasporto e messaggi

MCP usa messaggi JSON-RPC e mantiene il formato logico separato dal trasporto [21]. La specifica prevede, tra gli altri, processi locali collegati tramite standard input/output e server raggiunti via HTTP. I servizi GDACS e CEMS del progetto sono container autonomi e comunicano con il backend via HTTP.

3.3 Capacità esposte dai server

3.3.1 Tools

Un tool è una funzione descritta da un nome, da una spiegazione del comportamento e dallo schema degli argomenti. Nel server GDACS, ad esempio, la ricerca degli eventi recenti è esposta con un contratto equivalente al seguente:

```
tool: list_recent_events
input: { "hazard": "string", "window": "string" }
output: lista normalizzata di eventi GDACS
```

Descrizione e schema costituiscono le informazioni sulle quali il modello basa la scelta del tool e dei relativi parametri. Un contratto chiaro si traduce quindi direttamente in invocazioni più corrette.

Nel prototipo ogni tool corrisponde a un'operazione della sorgente, accetta un input validato e restituisce errori classificabili. Contratti circoscritti permettono inoltre di provare parsing e gestione dei guasti separatamente dal modello linguistico.

3.3.2 Resources e prompts

Una resource rende leggibile un contenuto indirizzabile, di norma identificato da un URI, mentre i prompts sono template testuali per interazioni ricorrenti. La prima descrive un dato da recuperare; i secondi forniscono all'host una procedura o un'istruzione

parametrica. Entrambe le capacità fanno parte della specifica; il flusso realizzato per GDACS e CEMS si basa sui soli tool.

3.4 Uso di MCP nel workflow

Nel progetto i client scoprono i tool all'apertura della sessione. Il ramo selezionato determina il server e il router specializzato sceglie l'operazione, così il modello vede solo i tool pertinenti. Dopo la validazione degli argomenti, il client invia la richiesta e il risultato normalizzato rientra nello stato LangGraph. La selezione è quindi distribuita fra decisione del modello e vincoli del grafo.

Quando l'orchestratore seleziona GDACS, il client MCP invia i parametri normalizzati al tool scelto e riceve una risposta strutturata. Il server si occupa della richiesta HTTP, del parsing XML e degli errori della sorgente; il workflow vede soltanto il contratto del tool. Lo stesso schema è usato per le attivazioni Copernicus EMS. MCP affianca quindi le API remote e ne isola i dettagli dal resto dell'applicazione.

3.5 Confronto e limiti

3.5.1 Integrazione diretta

Un server MCP, internamente, può chiamare qualunque API: REST, GraphQL, database SQL, SDK proprietari, comandi locali. Verso l'applicazione IA, invece, espone un'interfaccia uniforme.

Rispetto a un'integrazione collocata interamente nel backend, il costo aggiuntivo è la gestione di sessioni, trasporto e processi separati. In cambio, parsing XML di GDACS e accesso a CEMS restano confinati nei rispettivi servizi. Il client STAC, mantenuto direttamente nel backend, offre un termine di confronto concreto: è più semplice da eseguire, ma rende il backend responsabile dello schema e degli errori dell'API remota.

3.5.2 Tool calling

Il tool calling tradizionale descrive le funzioni che una singola applicazione rende disponibili al modello. MCP opera tra l'host e server separati, dai quali quelle funzioni

possono essere scoperte. L'host può quindi presentare al modello un tool ottenuto da un server MCP e tradurre l'invocazione nella corrispondente richiesta di protocollo. I due meccanismi intervengono in punti diversi della stessa catena e sono complementari.

3.5.3 Costi introdotti

Ogni sorgente conserva la propria semantica: eventi GDACS e attivazioni CEMS hanno campi, tempi di aggiornamento e condizioni di errore diversi. MCP introduce inoltre due servizi da distribuire e monitorare. Per un prototipo con poche funzioni tale costo è visibile; il vantaggio sta nel poter modificare una sorgente lasciando parsing e dipendenze fuori dal workflow.

3.6 Sicurezza e gestione operativa

3.6.1 Autenticazione e autorizzazione

Avvio dei server, accesso ai dati, gestione delle credenziali e revoca dei permessi sono responsabilità dell'host e dell'infrastruttura, più che del protocollo di integrazione.

I server sono raggiungibili sulla rete interna dei container e ricevono configurazione e credenziali dall'ambiente. Il server stabilisce le operazioni disponibili; il workflow limita ulteriormente quelle invocabili nel ramo corrente. Poiché tutti i tool sono di sola lettura, il prototipo può fare a meno di un flusso di conferma per azioni irreversibili.

3.6.2 Prompt injection e dati esterni

Feed e pagine web possono contenere testo che contraddice le istruzioni dell'applicazione. I contenuti recuperati vengono trattati come dati, mentre piano, tool disponibili, elenco degli indirizzi GDACS ammessi e schemi di output sono controllati dal codice. Questa separazione riduce la superficie d'attacco, pur senza escludere ogni forma di prompt injection.

3.6.3 Osservabilità

Il client registra servizio, durata, esito e codice diagnostico delle chiamate, omettendo credenziali e prompt completi. MCP offre un punto uniforme nel quale raccogliere queste informazioni, mentre log e metriche sono implementati dall'applicazione.

3.7 Criteri adottati nel progetto

I tool implementati sono piccoli, di sola lettura e dotati di schemi di input espliciti. Le credenziali e gli indirizzi interni dei servizi restano nella configurazione applicativa. I risultati includono invece URL pubblici delle risorse, identificativi e timestamp utili alla verifica. I test dei client coprono risposte vuote, payload malformati, timeout ed errori HTTP temporanei. Queste proprietà derivano dalla progettazione dei singoli server e dell'host, più che dall'adozione di MCP in sé.

Nel sistema proposto MCP ha un compito delimitato: separa i servizi GDACS e Copernicus EMS dal backend agentico e offre un'interfaccia uniforme per scoprirne e invocarne i tool. Il catalogo satellitare STAC, usato per acquisizioni e heatmap nel workspace, resta un'integrazione diretta, così come l'accesso della Discovery al catalogo NebulaGraph. La convivenza dei due approcci mostra in concreto quando MCP conviene come scelta architetturale.

Capitolo 4

Fonti informative geospaziali: GDACS e Copernicus

Il progetto integra fonti operative e cataloghi tematici Copernicus. GDACS, CDSE/STAC e CEMS On-demand Mapping descrivono eventi e allerte, acquisizioni satellitari e attivazioni cartografiche di emergenza. Marine e Drought ampliano la scoperta di risorse attraverso un catalogo di metadati a grafo. Ogni fonte risponde a una domanda diversa: una heatmap di acquisizioni, per esempio, mostra dove si concentrano le immagini disponibili, mentre una prima stima dell'impatto di un disastro viene dalle allerte GDACS. Il capitolo descrive quindi contenuto, modalità di accesso e limiti di ciascuna fonte prima di esaminare l'architettura che le combina.

4.1 GDACS

4.1.1 Scopo e contesto operativo

GDACS, acronimo di *Global Disaster Alert and Coordination System*, è un quadro di cooperazione tra le Nazioni Unite, la Commissione europea e gli attori internazionali della gestione delle emergenze. Il suo obiettivo principale è ridurre le carenze informative e di coordinamento nella prima fase successiva a grandi disastri improvvisi [12].

La piattaforma raccoglie dati sugli eventi, esegue o riceve stime automatiche dell'impatto potenziale e distribuisce notifiche. La sua funzione di *early awareness* mette in

relazione caratteristiche fisiche dell'evento, popolazione esposta, vulnerabilità e capacità di risposta. I modelli variano in base al tipo di pericolo e producono una valutazione sintetica utile a stabilire quali eventi richiedano attenzione internazionale [11].

4.1.2 Eventi e livelli di allerta

GDACS pubblica informazioni su diverse categorie di pericolo, tra cui terremoti, possibili tsunami, cicloni tropicali, alluvioni, eruzioni vulcaniche, siccità e incendi. I record disponibili contengono normalmente:

- identificativo e tipologia dell'evento;
- titolo, paese e coordinate;
- istante di pubblicazione o aggiornamento;
- livello di allerta;
- misure specifiche, come magnitudo o severità;
- collegamento alla scheda ufficiale.

I livelli verde, arancione e rosso esprimono una stima crescente dell'impatto e della rilevanza internazionale. Oltre alla misura fisica del fenomeno, il livello considera esposizione, vulnerabilità e capacità di risposta: un forte terremoto in un'area poco popolata può quindi ricevere un'allerta bassa.

4.1.3 Accesso programmatico

I dati e le stime GDACS sono accessibili tramite feed RSS e altri formati standard, così da poter essere integrati in portali e applicazioni esterne [12]. Sono disponibili feed distinti per intervallo temporale e tipologia di evento; la documentazione indica, ad esempio, feed relativi alle ultime 24 ore, all'ultima settimana e a specifiche soglie per i terremoti [10].

Nel progetto il feed viene interrogato da un servizio MCP autonomo, che normalizza il contenuto XML e restituisce oggetti strutturati. L'agente riceve così un contratto

stabile e filtra i record per luogo, periodo, tipo di evento e, quando richiesto, soglie quantitative.

4.1.4 Limiti

Una notifica GDACS è una stima rapida e può cambiare quando arrivano nuovi dati; le valutazioni sul campo e i bollettini ufficiali di protezione civile restano il riferimento definitivo. Alcuni processi sono automatici, altri includono dati e valutazioni provenienti da istituzioni o fonti esterne. Per questo il progetto conserva link, timestamp e identificativi: ogni risposta dell'agente resta verificabile e presenta le stime preliminari come tali.

4.2 Il programma Copernicus

Copernicus è il programma di osservazione della Terra dell'Unione europea. Mette a disposizione missioni satellitari, dati, infrastrutture di accesso e servizi tematici. Per questa tesi sono rilevanti percorsi complementari:

1. il *Copernicus Data Space Ecosystem* (CDSE), usato per cercare metadati relativi alle acquisizioni satellitari;
2. il *Copernicus Emergency Management Service* (CEMS), usato per consultare attivazioni e prodotti cartografici di emergenza;
3. Copernicus Marine, per individuare prodotti, dataset e variabili oceanografiche;
4. i servizi Drought consultati dall'harvester, per scoprire layer, coverage e disponibilità temporali relativi alla siccità.

Il catalogo satellitare descrive osservazioni disponibili; CEMS documenta invece attivazioni operative e prodotti derivati dall'analisi di immagini e di altre informazioni.

4.3 Copernicus Data Space Ecosystem e STAC

4.3.1 Ricerca delle acquisizioni

Il CDSE offre API per catalogazione, download, visualizzazione e processamento dei dati di osservazione della Terra. Fra le API di catalogo è disponibile STAC, insieme a OData e Sentinel Hub Catalog API [4].

STAC, *SpatioTemporal Asset Catalog*, definisce un modello interoperabile per descrivere collezioni e singoli elementi geospaziali. Un elemento associa un identificativo a geometria, intervallo temporale, proprietà, collegamenti e asset [36]. La ricerca può quindi combinare:

- una o più collezioni, ad esempio Sentinel-2 Level-2A;
- un intervallo temporale;
- un'area espressa come bounding box;
- filtri su attributi, quali copertura nuvolosa o piattaforma;
- un limite al numero di risultati.

Il catalogo CDSE implementa STAC 1.1.0 ed espone un endpoint di ricerca standardizzato [5].

4.3.2 Dal metadato alla heatmap

Il ramo STAC del progetto lavora sui metadati: recupera le acquisizioni che intersecano luogo e periodo richiesti, senza analizzare il contenuto delle immagini. Di ogni prodotto si usa il rettangolo che lo racchiude (bounding box), se ne calcola il centro e si costruisce:

- una heatmap che mostra dove si concentrano le acquisizioni recuperate;
- i rettangoli delle bounding box, selezionabili;
- un pannello con identificativo, collezione, data, piattaforma, copertura nuvolosa, asset e collegamenti disponibili.

I conteggi mostrati si riferiscono ai prodotti restituiti dalla ricerca: più prodotti possono derivare dalla stessa ripresa satellitare, ad esempio a diversi livelli di elaborazione. La visualizzazione descrive quindi la disponibilità di dati sull'area, come punto di partenza per analisi successive.

4.4 Copernicus Emergency Management Service

CEMS fornisce informazioni geospaziali a supporto della risposta alle emergenze e della gestione del rischio. È uno dei sei servizi Copernicus e comprende monitoraggio e allerta precoce, cartografia on demand e informazioni sull'esposizione [6].

Nel progetto viene usata la componente *On-demand Mapping*. Le attivazioni sono identificate da codici come `EMSR884` o `EMSN204`. Il prefisso consente di distinguere due percorsi principali:

Rapid Mapping (`EMSR`). Produce in tempi rapidi informazioni geospaziali per la fase immediatamente successiva a un evento. Una attivazione può comprendere più aree di interesse e più prodotti [7].

Risk and Recovery Mapping (`EMSN`). Supporta prevenzione, preparazione e ricostruzione mediante analisi geospaziali più articolate, basate su immagini ottiche e radar [8].

Il relativo server MCP espone operazioni per elencare le attivazioni, ottenere una sintesi e recuperare i dettagli di una specifica attivazione. I risultati normalizzati includono codice, nome, categoria, paesi, fase di gestione del rischio, stato, data, aree di interesse e prodotti disponibili.

4.5 Marine e Drought nel catalogo di Discovery

La richiesta di una risorsa scientifica richiede metadati più articolati del solo elenco delle acquisizioni. Per esempio, studiare la temperatura superficiale del Mar Nero implica individuare un dataset, la variabile disponibile, l'area coperta e la risoluzione temporale. La Discovery usa un catalogo locale alimentato da tre harvester, documentati nella guida tecnica del progetto [29].

4.5.1 Metadati Marine

Il collector Marine legge il catalogo completo di Copernicus Marine, comprese le versioni precedenti dei dataset. Conserva la gerarchia tra prodotto, dataset, versione, parte e servizio di accesso, oltre a variabili e coordinate. Una variabile è descritta da nome, nome standard, unità e descrizione; gli assi da estremi, passo e numero di valori, senza memorizzare i valori completi.

Un secondo collector documenta le operazioni offerte dalla libreria ufficiale di Copernicus Marine e i relativi parametri. Il catalogo può così indicare, per ogni risorsa, le modalità con cui sarebbe possibile estrarne una porzione; l'harvester registra queste possibilità senza scaricare i file scientifici.

4.5.2 Metadati Drought

Il collector Drought interroga le descrizioni standard dei servizi cartografici WMS e WCS. Raccoglie servizi, layer e sotto-layer, coverage ed estensioni temporali. Queste descrizioni consentono di verificare se una risorsa sia compatibile con tema e periodo richiesti senza scaricare mappe o raster. Il censimento copre le coverage previste dal catalogo applicativo.

4.5.3 Due usi distinti di STAC

L'harvester CDSE salva root del catalogo, collezioni, campi interrogabili e, quando previsto, item e riferimenti agli asset. La finestra recente predefinita per gli item è di due giorni, con limiti configurabili. Degli asset si conservano i collegamenti, mentre i file restano sul servizio remoto.

Nella Discovery l'agente legge questi metadati da NebulaGraph; nel percorso operativo il client interroga direttamente CDSE STAC per cercare acquisizioni. I due percorsi sono indipendenti: i risultati operativi riflettono lo stato attuale del servizio remoto, mentre il grafo riflette l'ultimo aggiornamento del catalogo locale.

Aspetto	GDACS	CDSE/STAC	Copernicus CEMS
Domanda	Quali eventi richiedono attenzione?	Quali acquisizioni satellitari coprono area e periodo?	Quali attivazioni e prodotti di emergenza sono disponibili?
Unità	Evento e allerta	Item e collezione satellitare	Attivazione, area di interesse e prodotto
Uso	Monitoraggio di eventi	Heatmap e bounding box dei prodotti	Prodotti Rapid Mapping e Risk/Recovery
Integrazione	Server MCP GDACS	Client STAC nel backend	Server MCP CEMS

Tabella 4.1: Ruoli delle fonti del percorso operativo. Marine e Drought alimentano inoltre il catalogo di Discovery descritto nella Sezione 4.5.

4.6 Confronto e complementarità

Le tre sorgenti rispondono a domande diverse e possono essere combinate. Per una richiesta su un'alluvione, GDACS fornisce gli eventi e il livello di allerta; CEMS può indicare un'attivazione di cartografia; STAC può mostrare quali acquisizioni satellitari sono presenti nell'area e nel periodo. Una ricerca web aggiunge infine contesto recente e fonti narrative.

4.7 Implicazioni per la progettazione dell'agente

I server MCP e il client STAC convertono feed XML, risposte STAC e record CEMS in dataset con campi stabili. In questo modo i client possono essere provati separatamente dal modello, i rami indipendenti possono procedere in parallelo e il guasto di una sorgente lascia intatti i risultati già disponibili.

Il workflow conserva semantica e provenienza di ogni record anche quando GDACS, CDSE/STAC e CEMS sono interrogati nello stesso turno. Allo stesso modo, il grafo mantiene distinti dataset, variabile, collezione e layer: una risorsa scoperta indica quali dati potrebbero sostenere uno studio, il cui risultato richiede poi un'analisi dedicata. Su questa separazione si basa l'architettura del capitolo successivo.

Capitolo 5

Architettura del sistema proposto

5.1 Obiettivo e requisiti

Il sistema proposto è un assistente conversazionale specializzato in geografia, osservazione della Terra ed emergenze naturali, esposto come *Copernicus Data Workspace*. Interpreta richieste in linguaggio naturale, interroga più fonti, produce testo e visualizzazioni e mantiene il contesto nei turni successivi, anche riprendendo sessioni già avviate.

I requisiti che guidano l'architettura sono:

- autenticare l'accesso al workspace (OIDC) e associare memoria e artefatti alla sessione di lavoro;
- applicare un confine di dominio prima di effettuare chiamate esterne;
- selezionare una o più fonti in funzione della richiesta;
- distinguere la Discovery sul catalogo a grafo, la ricerca STAC sul catalogo remoto e, quando serve, il percorso combinato;
- eseguire in parallelo i rami operativi indipendenti;
- mantenere separati ragionamento, acquisizione dati e presentazione;
- conservare memoria e artefatti per follow-up e ripresa sessioni;

- presentare le acquisizioni Copernicus in una dashboard interattiva e modificabile, lasciando intatti i dati di origine;
- continuare a fornire un risultato quando una sola sorgente fallisce;
- esporre un'API compatibile con OpenAI Chat Completions (SSE), usata dal workspace e riutilizzabile da client legacy;
- rendere osservabili errori, tempi e dipendenze;
- consentire la valutazione del workflow con riferimenti verificabili, controlli oggettivi e rubriche esplicite.

5.2 Componenti applicativi

L'accesso alle fonti geospaziali è mediato da un workspace conversazionale e da un workflow persistente. I componenti vengono qui raggruppati per funzione; le dipendenze e i nodi del grafo sono descritti nelle sezioni seguenti.

5.2.1 Interfaccia e API

L'interfaccia primaria è *Copernicus Data Workspace*, servita direttamente dal backend. È uno spazio di lavoro a schermo intero, con home in italiano e selettore IT/EN, in cui l'utente:

- formula la domanda in linguaggio naturale (Discovery, analisi operativa o percorso combinato);
- avvia una nuova indagine oppure riprende una sessione recente che conserva filtri, artefatti e contesto;
- consulta testo, tabelle e dashboard generate dal workflow.

L'autenticazione avviene tramite Keycloak (OIDC). I ruoli distinguono almeno un profilo amministratore, con accesso anche agli strumenti di valutazione, e un profilo utente standard.

Open WebUI resta disponibile come client legacy opzionale: inoltra la chat al backend nel formato delle completion OpenAI [25]. Un viewer cartografico STAC completa l'offerta come vista dedicata, distinta dalla dashboard analitica.

FastAPI espone l'endpoint di chat, le API degli artefatti (lettura, modifiche, revisioni, chat contestuale) e lo streaming SSE. I modelli Pydantic validano richieste, decisioni degli agenti e risultati normalizzati prima che passino al nodo successivo [31]. Il formato HTTP resta così separato dalla rappresentazione interna dello stato.

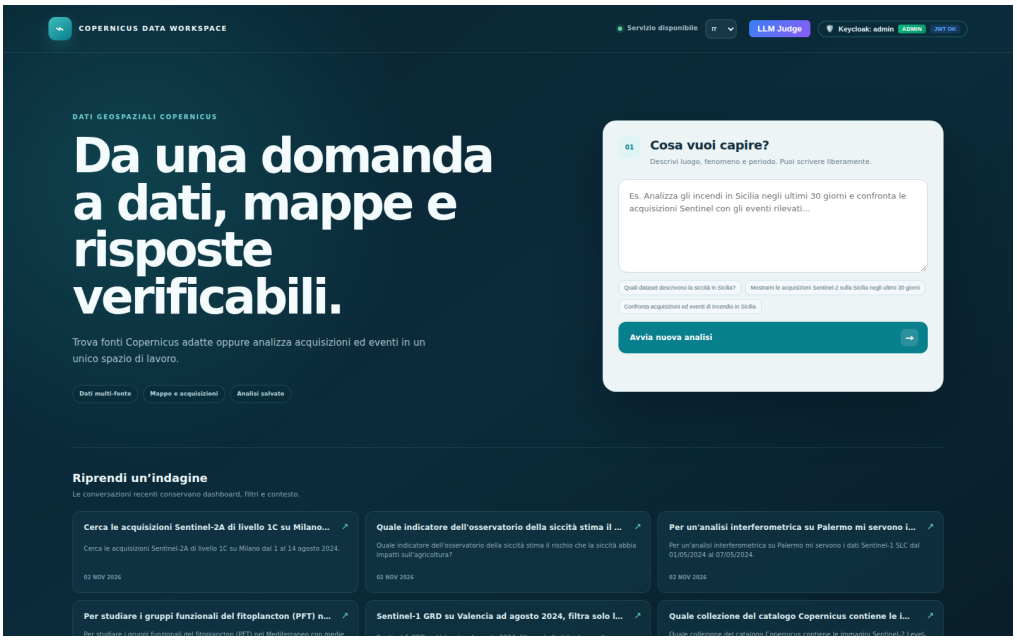


Figura 5.1: Home della dashboard nativa (*Copernicus Data Workspace*): ingresso della domanda, suggerimenti, ripresa delle indagini e stato del servizio autenticato con Keycloak.

5.2.2 Orchestrazione e memoria

LangChain gestisce messaggi, invocazioni e output strutturati del modello [15]; LangGraph dispone le operazioni in un grafo con rami condizionali e punti di convergenza [16]. Il modello propone il piano; il grafo stabilisce quali nodi eseguire e quando i risultati sono pronti per la risposta.

Redis conserva il checkpoint associato alla conversazione: messaggi utili, risultati raccolti, filtri e riferimenti agli artefatti. Un follow-up come "raggruppali per giorno"

opera quindi sul dataset precedente senza interrogare di nuovo le sorgenti. La persistenza per conversazione deriva da LangGraph e dalla sua integrazione con Redis [17, 32]; il riuso selettivo dei risultati è una scelta del workflow applicativo [29]. La chiave della memoria è l'identificativo della sessione del workspace (o, nel client legacy, gli identificativi di utente e chat).

5.2.3 Modello e distribuzione

Il modello linguistico è servito da Ollama su una macchina distinta dal backend (nel prototipo, tipicamente qwen3:8b per il workflow e un modello separato per il Judge) [24]. Riceve il contesto preparato dai nodi e interviene nell'interpretazione, nel piano e nella risposta; l'accesso alle fonti, alla memoria e il calcolo dei valori restano compito del codice applicativo, che gestisce le chiamate con timeout, retry selettivi e limiti di concorrenza [29].

Docker Compose coordina backend, interfaccia, Keycloak, Redis, servizi MCP e NebulaGraph su una rete condivisa; configurazioni opzionali aggiungono il client legacy, la raccolta periodica dei metadati e l'osservabilità [9, 29]. La Tabella 5.1 riporta la responsabilità assegnata a ciascun elemento dello stack.

Tecnologia	Ruolo nel sistema
Copernicus Data Workspace	Home, chat, artefatti Discovery/operativi e ripresa sessioni.
Keycloak	Autenticazione OIDC e ruoli applicativi.
LangChain	Comunicazione strutturata tra agenti e modello linguistico.
LangGraph	Coordinamento dei passaggi, delle fonti e dei follow-up.
Redis Stack	Checkpoint per conversazione e riuso dei risultati.
NebulaGraph	Catalogo condiviso di metadati Marine, STAC e Drought.
Ollama	Inferenza del modello usato dal workflow (e, se configurato, dal Judge).
FastAPI e Pydantic	API del sistema e controllo della forma dei dati scambiati.
Docker Compose	Avvio coordinato dei servizi su una rete condivisa.
Prometheus e Grafana	Raccolta delle metriche e diagnosi aggregata LLMOps.
OpenTelemetry e Phoenix	Tracciamento delle singole esecuzioni.
Open WebUI (opzionale)	Client legacy di chat.

Tabella 5.1: Tecnologie del nucleo applicativo, del catalogo e dell'osservabilità.

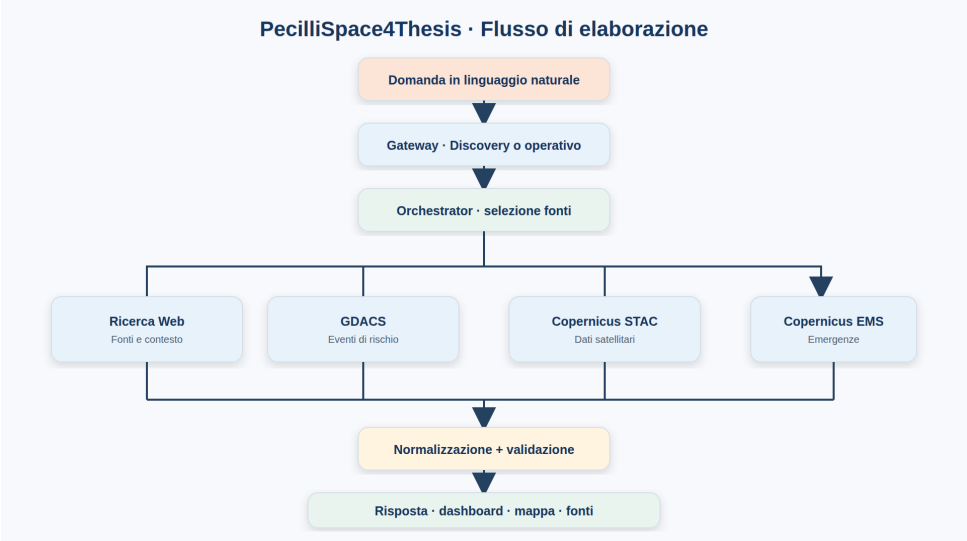


Figura 5.2: Flusso di elaborazione sintetico: dalla domanda in linguaggio naturale alla risposta con dashboard, mappa e fonti. Il gateway distingue Discovery e percorso operativo prima della selezione delle sorgenti.

5.3 Vista architetturale

L'applicazione segue una struttura a componenti. La dashboard nativa gestisce l'interazione con l'utente (chat, ripresa sessioni, artefatti Discovery e acquisizioni) dopo l'accesso tramite Keycloak; FastAPI espone gli endpoint della chat e degli artefatti; LangGraph coordina gli agenti; i servizi MCP e i client dedicati accedono alle fonti esterne. La dashboard analitica delle acquisizioni è un artefatto HTML che il browser carica tramite API e costruisce a partire da un contratto JSON validato.

Il catalogo NebulaGraph aggiunge una base di metadati condivisa fra le conversazioni, distinta dai checkpoint Redis. Gli harvester aggiornano il catalogo; il backend lo consulta attraverso query predefinite. Metriche e trace osservano l'esecuzione, mentre il runner del benchmark invoca lo stesso workflow per valutarlo [29].

Il modello linguistico produce decisioni strutturate; il codice applicativo le valida, invoca le integrazioni e ne acquisisce il risultato.

In questa architettura LangGraph e Redis costituiscono il nucleo con stato, FastAPI espone il servizio e Pydantic delimita i contratti (Tabella 5.1 e Figura 5.4); ECharts e Leaflet sono le librerie di presentazione usate dal browser.

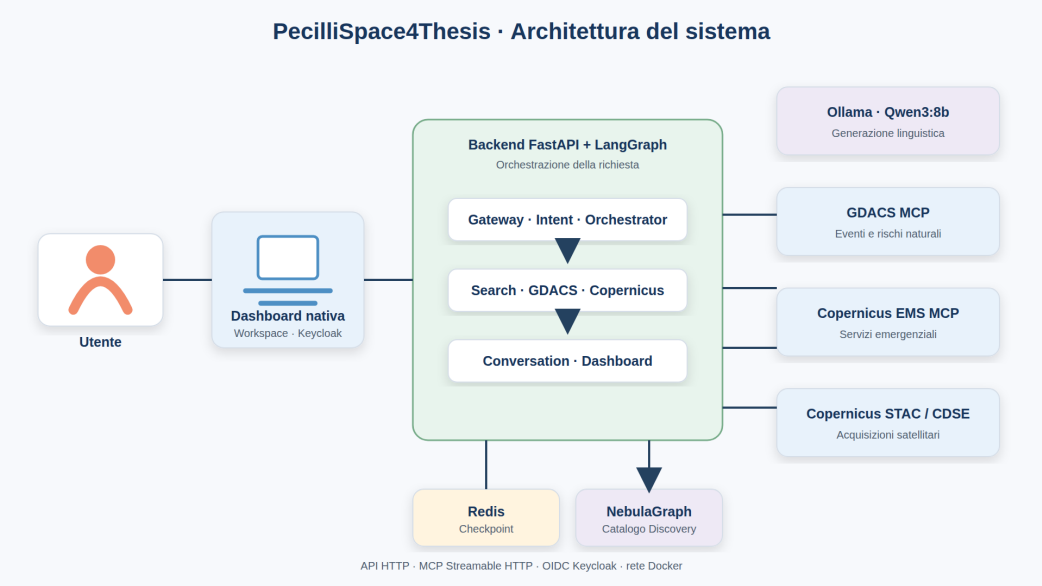


Figura 5.3: Vista a componenti del prototipo: dashboard nativa autenticata, backend LangGraph, Redis, NebulaGraph e sorgenti MCP/STAC sulla rete Docker condivisa.

5.4 Organizzazione del backend e del frontend

Il backend divide le responsabilità in moduli distinti: gli agenti del workflow (gateway, interpretazione, orchestrazione, fonti, Discovery, grafici e conversazione); le API per chat, salute del sistema, artefatti e valutazioni; un nucleo comune con stato condiviso, gestione degli errori e osservabilità; i client verso le fonti e verso il catalogo a grafo; i modelli dei dati scambiati; i servizi di supporto (risoluzione di luogo e tempo, artefatti, health check); gli strumenti di valutazione. Un modulo dedicato costruisce il grafo LangGraph.

Il frontend è servito dal backend e separa chat, workspace, filtri, grafici, mappa, tabelle, revisioni, confronti e pannello valutazioni. Il browser usa ECharts per i grafici e Leaflet per la mappa; l'interfaccia è bilingue (IT/EN). Il viewer STAC dedicato è un'applicazione separata.

Strumentazione, harvester e schema del catalogo hanno pacchetti propri. I due server MCP sono esterni all'applicazione, con dipendenze e container autonomi: un problema del feed GDACS resta confinato al relativo servizio, senza toccare i rami CEMS o STAC.

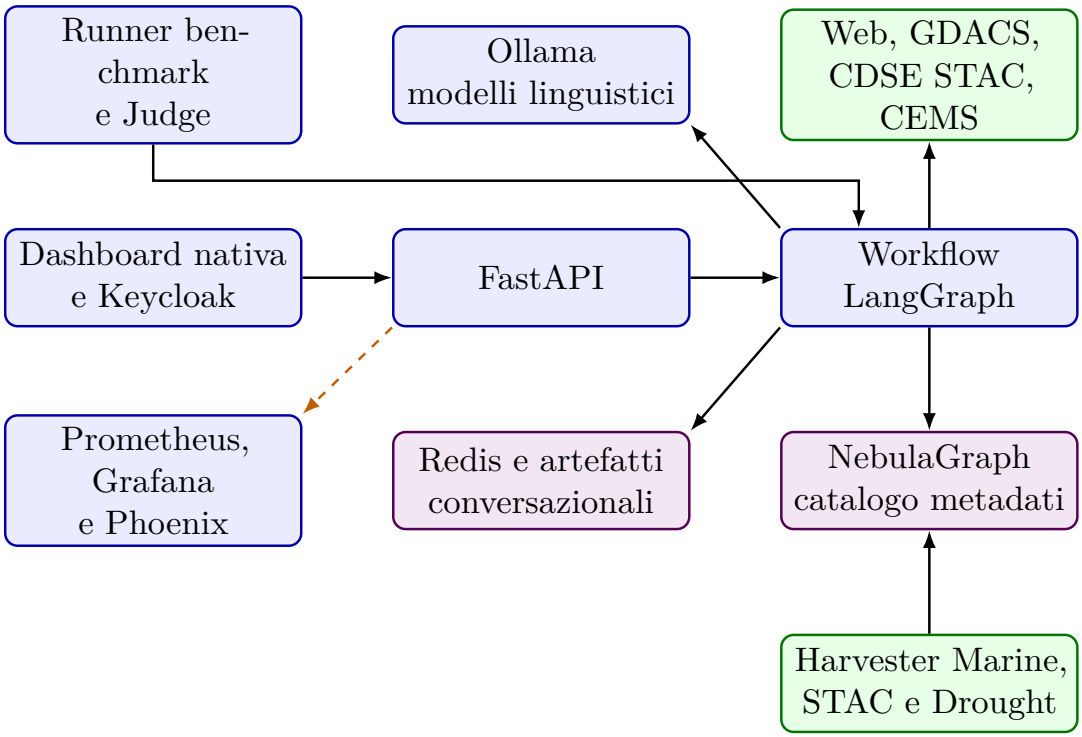


Figura 5.4: Componenti applicativi, catalogo e strumenti di analisi. La Discovery legge NebulaGraph; il ramo operativo consulta le fonti remote. Il benchmark invoca il workflow, mentre la telemetria ne osserva l'esecuzione.

5.5 Il grafo degli agenti

5.5.1 Percorso principale

Il workflow ha un unico punto di ingresso. Una nuova richiesta attraversa una parte comune e viene instradata verso la Discovery o verso il percorso operativo; in quest'ultimo il piano seleziona le fonti necessarie.

Nel ramo operativo Search e Copernicus STAC costituiscono il piano di base; GDACS e CEMS lo integrano secondo la richiesta. LangGraph avvia un'istanza di lavoro per ogni sorgente selezionata, così Search, GDACS e STAC procedono in parallelo. Quando sono richiesti insieme CEMS e STAC, il sistema esegue prima CEMS: l'area o le coordinate dell'attivazione rendono più precisa la successiva ricerca satellitare.

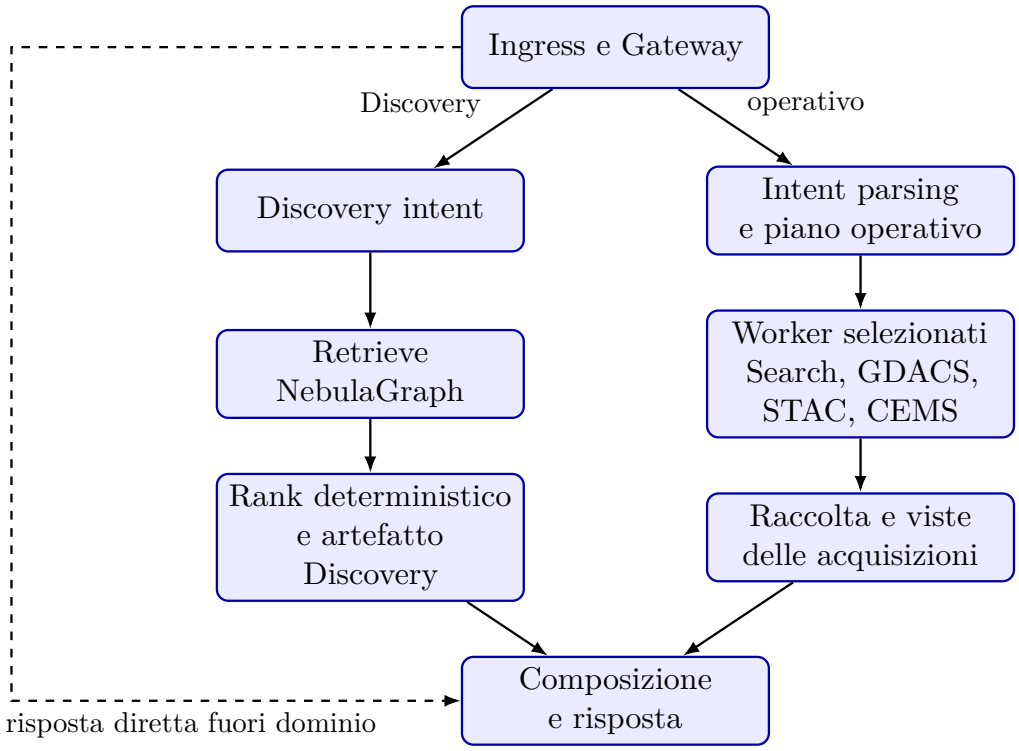


Figura 5.5: Due percorsi di una nuova richiesta ammessa. Il disegno sintetizza i rami; follow-up, nodi finali di verifica e dipendenza CEMS–STAC sono descritti nel testo.

5.5.2 Gateway e confine di dominio

Il Gateway classifica ogni turno della conversazione. Sono ammessi argomenti relativi a geografia, spazio, osservazione della Terra, clima, pericoli naturali ed emergenze. Una domanda estranea riceve subito una risposta diretta, senza attivare parser, ricerca o dashboard.

Se la classificazione fallisce, il sistema adotta un comportamento *fail closed* e resta senza chiamate esterne. La scelta riduce le chiamate superflue e impedisce che un errore del modello allarghi implicitamente il dominio operativo.

5.5.3 Parsing dell'intento

L'agente di interpretazione converte il testo in una richiesta strutturata che contiene:

- argomenti e relativa categoria;

- luoghi testuali, paese, città, coordinate o bounding box;
- intervallo temporale normalizzato e fuso orario;
- obiettivo dell'analisi, filtri e aggregazioni;
- modalità di risposta e viste richieste.

Espressioni quali "oggi", "la settimana scorsa" o "negli ultimi 30 giorni" vengono trasformate in estremi temporali. Se manca un'informazione indispensabile, il parser chiede un chiarimento; la domanda parziale viene conservata e completata al turno successivo.

5.5.4 Orchestratore e piano di esecuzione

L'Orchestratore restituisce una lista di sorgenti scelte fra ricerca web, GDACS, Copernicus STAC e CEMS. Alla decisione del modello si affiancano regole deterministiche: nel percorso operativo Search e Copernicus STAC forniscono contesto web e acquisizioni per la dashboard; le richieste su pericoli naturali aggiungono GDACS; un codice di attivazione CEMS esplicito aggiunge il ramo dedicato. Il workflow usa inoltre GDACS come fonte strutturata di supporto quando il piano segnala un possibile pericolo.

Ne risulta un'orchestrazione ibrida: l'LLM gestisce la varietà del linguaggio, regole verificabili preservano i vincoli applicativi.

5.5.5 Il ramo Discovery

La Discovery procede in tre passi: interpretazione, recupero e ordinamento. L'agente d'intento rappresenta tema, area, periodo, tipo di risorsa e variabili. Il servizio di Discovery espande alias di dominio, seleziona i cataloghi e interroga il grafo; controlli di compatibilità filtrano i candidati sulla base dei metadati disponibili. Il luogo richiesto viene rispettato, e ogni alternativa temporale proposta è accompagnata da un avviso.

Il ranking è basato su regole e calcolato dal codice. Il punteggio combina caratteristiche verificabili nei metadati (corrispondenza di titolo, identità e variabili, compatibilità spaziale e temporale, tipo di risorsa, servizio preferito), applica penalità per le incompatibilità e viene normalizzato in $[0, 1]$. Sono selezionati i candidati con punteggio almeno

$\max(0{,}28, \text{best} - 0{,}20)$, fino a 30 risorse; se la domanda nomina più variabili, ciascuna ha almeno un rappresentante. Le regole sono semplici e ispezionabili, ma fissate a mano: come mostrano le metriche di ordinamento del Capitolo 8, il ranking è il punto più debole della Discovery.

Ogni interrogazione al grafo restituisce al massimo 120 righe, e al ranking arrivano al più 30 candidati: qualità del recupero e qualità dell'ordinamento vanno quindi valutate separatamente. Il benchmark usa spesso un solo target per domanda, mentre una richiesta di Discovery può ammettere più risorse valide; una risorsa corretta diversa dal target può perciò risultare penalizzata. La risposta e l'artefatto riportano identificativi, metadati e collegamenti delle risorse selezionate [29].

5.6 Catalogo a grafo e pipeline di harvesting

5.6.1 Modello del dominio

NebulaGraph conserva vertici, archi orientati e proprietà. I tag descrivono i tipi dei vertici; gli edge type descrivono le relazioni. Un nodo radice Copernicus riunisce i tre cataloghi Marine, CDSE/STAC e Drought.

Sottografo	Entità e relazioni principali
Marine	Product, Dataset, Version, Part, DataService, Variable e Coordinate; Operation e Parameter documentano l'accesso.
CDSE/STAC	StacCatalog, StacCollection, StacQueryable, StacItem e StacAsset; gli asset conservano riferimenti ai file.
Drought	OgcService, DroughtLayer, DroughtCoverage e DroughtTimeExtent; sono rappresentate gerarchie dei layer e disponibilità temporali.

Tabella 5.2: Articolazione del catalogo condiviso di metadati.

Ogni vertice ha un identificativo stabile con un prefisso che ne indica il catalogo di provenienza. Un dataset Marine è collegato alle sue variabili sia direttamente sia attraverso versioni e servizi di accesso. La radice comune consente di navigare fra i cataloghi; le corrispondenze tra variabili Marine e indicatori Drought restano un possibile sviluppo.

5.6.2 Raccolta, aggiornamento e riconciliazione

Ogni harvester definisce lo schema del proprio sottografo, raccoglie e normalizza i metadati e li scrive nel grafo aggiornando o creando vertici e archi.

Per ogni risorsa il catalogo registra prima osservazione, ultima presenza e ultima elaborazione, oltre a un'impronta dei metadati che rivela i cambiamenti. Le risorse scomparse vengono marcate come inattive anziché cancellate; una vista corrente filtra quindi vertici e archi attivi.

Un harvest che farebbe sparire più di un quarto delle risorse di una categoria viene trattato come sospetto. Gli item STAC sono raccolti per finestre temporali successive, e l'assenza da una finestra recente vale solo per quella finestra. Il raggiungimento di un limite di sicurezza è segnalato come errore.

La scrittura avviene a blocchi: un guasto a metà raccolta può lasciare salvati alcuni blocchi. Il grafo conserva lo stato corrente delle proprietà; timestamp e indicatore di attività rendono lo stato ispezionabile, mentre la storia completa delle revisioni richiederebbe un meccanismo di versionamento dedicato.

Gli harvester vengono eseguiti ogni notte, con lock, timeout e tentativi limitati. La salute del catalogo considera sia la raggiungibilità del database sia la freschezza dell'ultima raccolta riuscita [29].

5.6.3 Infrastruttura e confine di lettura

NebulaGraph separa gestione dello schema, conservazione dei dati ed esecuzione delle query in tre servizi distinti [22]. Il prototipo usa NebulaGraph 3.8.0 con un'istanza per servizio e una sola replica dei dati, una configurazione adatta allo sviluppo; l'alta disponibilità richiede repliche aggiuntive.

La Discovery usa soltanto query predisposte dal codice. Il client di lettura accetta esclusivamente istruzioni di consultazione, blocca istruzioni multiple e modifiche e valida i parametri; il modello linguistico fornisce parametri, mai query complete. Il vincolo è applicato dall'applicazione: l'utente del database configurato nel prototipo ha privilegi completi.

5.7 Agenti specializzati e nodi di elaborazione

Nel codice molti componenti si chiamano "Agent", ma solo alcuni usano un LLM per decidere. La Tabella 5.3 distingue i nodi decisionali linguistici dai servizi e dalle trasformazioni deterministiche.

Componente	LLM?	Decide cosa fare?	Operazione principale
Gateway	Sì	Sì (dominio/route)	Classificazione della richiesta
Discovery intent	Sì	Sì (intento catalogo)	Estrazione tema/area/periodo
Intent / Orchestrator	Sì	Sì (piano fonti)	Parametri e selezione sorgenti
Discovery retrieve	No	No	Query parametrizzate sul grafo
Discovery rank	No	Secondo regole fisse	Ordinamento a punteggio
Worker fonti	Ibrido	Esecuzione del piano	Recupero Search, GDACS, STAC e CEMS
Chart	Opzionale	Configurazione viste	Aggregazioni da record e rifinitura della vista
Validator	No	No	Controlli strutturali sulle viste
Conversation	Sì	Sì (sintesi/follow-up)	Composizione della risposta

Tabella 5.3: Componenti del workflow: uso di LLM, potere decisionale e operazione. "Ibrido" indica un nodo che può usare il modello per parametri, ma esegue recupero e filtri nel codice.

Gli agenti decisionali linguistici sono quindi concentrati in gateway, interpretazione, orchestrazione e conversazione; ranking, accesso al grafo, filtri STAC e validazione delle viste sono elaborazioni deterministiche.

5.8 Contratti e stato condiviso

Gli agenti decisionali producono output strutturato validato da Pydantic. Questa scelta separa due linguaggi:

- JSON tipizzato per lo scambio interno;

- Markdown in linguaggio naturale per l'utente.

Lo stato del grafo contiene messaggi, richiesta risolta, piano, azione di follow-up, risultati delle quattro fonti, grafici, dashboard e risposta finale. Per la dashboard conserva inoltre filtri collegati, stato visuale, revisioni e conversazioni contestuali per settore. Ogni stato è associato all'identificativo della conversazione, che separa logicamente i checkpoint.

Il ramo Discovery aggiunge intento, risorse recuperate e selezionate e riferimento al proprio artefatto. Questi dati appartengono alla singola conversazione; il catalogo NebulaGraph è invece una base persistente condivisa, aggiornata dagli harvester.

I risultati delle sorgenti seguono un formato comune: stato dell'operazione, dataset normalizzato, aggregazioni, fonti ed eventuali errori. L'agente di conversazione lavora così su un'unica rappresentazione, indipendente dai formati originali di RSS, STAC o CEMS.

5.9 Persistenza

La persistenza risponde a due bisogni: riprendere una conversazione e conservare artefatti troppo grandi per il prompt.

5.9.1 Checkpoint conversazionali

Redis Stack è l'unico backend della memoria conversazionale. LangGraph salva messaggi e stato dopo i nodi del grafo e, a ogni nuova richiesta, ricarica automaticamente il checkpoint della conversazione. Il backend si dichiara pronto solo quando Redis è raggiungibile.

La cronologia attiva è limitata agli ultimi messaggi utili, mentre i risultati strutturati restano nello stato: il prompt mantiene così una dimensione contenuta.

5.9.2 Archivio degli artefatti

Le heatmap e i payload della dashboard possono essere più grandi della memoria testuale e sono quindi salvati come artefatti JSON in un archivio dedicato. Lo stato conserva l'identificativo; il browser legge l'artefatto tramite API e invia modifiche controllate

a endpoint dedicati. Ogni modifica crea una revisione che può essere confrontata, annullata o ripristinata. Se CDSE è temporaneamente indisponibile, il servizio può riusare un artefatto precedente compatibile per area, collezione e intervallo temporale, dichiarandolo nella risposta.

5.10 Dashboard generativa e verificabile

Nel percorso operativo il workflow costruisce la dashboard analitica dai soli record STAC Copernicus normalizzati; nel workspace essa compare come artefatto del percorso "Analisi dati Copernicus". La Discovery presenta invece risorse del catalogo a grafo ("Ricerca dati Copernicus"). Search, GDACS e CEMS alimentano la risposta testuale o sezioni dedicate. Ogni serie quantitativa ha così un'unica provenienza, e lo stesso grafico confronta solo grandezze omogenee.

L'artefatto contiene KPI, grafici a barre, linee o torta, tabella, heatmap e provenienza: campi usati, filtri attivi, metodo di aggregazione, numero di record e identificativi dei prodotti. L'LLM propone le viste; conteggi, percentuali, medie, variazioni e confronti sono calcolati dal codice sui record recuperati.

I controlli consentono di cambiare tipo di grafico, ordinamento, Top N, visibilità e filtro. Un click su grafico o tabella può attivare un cross-filter: backend e interfaccia ricalcolano coerentemente grafici, KPI, tabella e heatmap. La chat contestuale di ciascun settore risponde sul risultato corrente e può richiedere una modifica tipizzata; ricorre alla ricerca web solo quando la domanda richiede contesto esterno ai dati visualizzati.

5.10.1 Leggibilità e correttezza dei grafici

La dashboard applica regole di impaginazione che mantengono leggibili anche i grafici più densi. I grafici temporali, quelli con più di otto categorie e il grafico primario occupano l'intera riga; gli altri sono disposti in una griglia responsiva. Per molte categorie sono disponibili scorrimento e zoom; le etichette vengono troncate, ruotate quando necessario e distanziate. Le torte mostrano etichette esterne fino a sei categorie e usano una legenda scorrevole; le barre orizzontali adattano l'altezza al numero di voci. I grafici si ridimensionano con il contenitore e quando la sezione viene riaperta.

Prima di salvare una modifica, il Validator controlla l'allineamento di etichette, valori e serie. Nella versione attuale sostituisce con zero i valori mancanti o non numerici e segnala la riparazione: il grafico resta disegnabile, ma un dato assente appare come uno zero osservato. Una miglioria naturale è escludere tali valori e mostrarli come mancanti. Ogni grafico offre anche una tabella dei dati e l'esportazione CSV, così il risultato visivo resta verificabile.

5.11 Integrazione MCP

I server GDACS e CEMS usano FastMCP e il trasporto Streamable HTTP. Il backend scopre e invoca i tool tramite un client comune. Gli strumenti principali sono:

- elenco degli eventi GDACS recenti, filtrato per pericolo e finestra;
- dettaglio di un evento GDACS ufficiale;
- catalogo delle operazioni CEMS;
- elenco, sintesi e dettaglio delle attivazioni CEMS.

Indirizzi interni e credenziali restano nella configurazione dei servizi, fuori dalla portata del modello; nei risultati compaiono solo i collegamenti pubblici ai prodotti o alle schede ufficiali. Il client applica timeout; i servizi ripetono le richieste solo per errori transitori.

5.12 Distribuzione tramite container

La distribuzione comprende servizi applicativi e catalogo condiviso:

1. dashboard nativa (asset statici serviti dal backend);
2. Keycloak per l'autenticazione OIDC;
3. backend FastAPI;
4. server MCP GDACS;

5. server MCP CEMS;

6. Redis Stack;

7. NebulaGraph con i suoi tre servizi e un passo di inizializzazione dello storage.

Ollama è raggiunto come servizio esterno. I server MCP sono visibili solo sulla rete interna; l'utente accede alla dashboard dal browser e, se attivato, al client legacy Open WebUI. L'ordine di avvio è governato da dipendenze e, per i servizi predisposti, da health check. Configurazioni aggiuntive abilitano la raccolta notturna dei metadati e l'osservabilità (Prometheus, Grafana e Phoenix). Liveness e readiness sono separate: un processo attivo è pronto solo quando dispone della propria memoria.

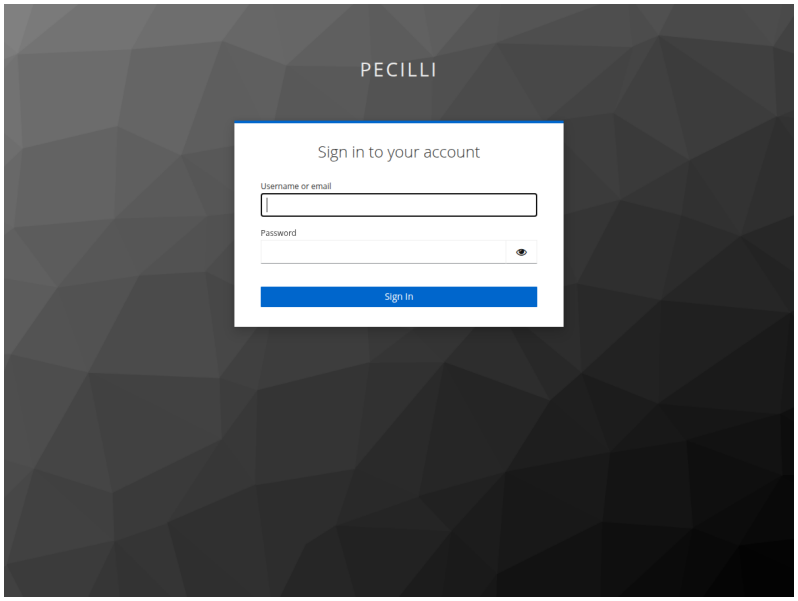


Figura 5.6: Schermata di accesso Keycloak, intermedia fra il browser e la dashboard nativa.

5.13 Sicurezza e osservabilità

Il prototipo adotta le seguenti misure per l'interfaccia e l'accesso alle fonti esterne:

- accesso all'interfaccia tramite Keycloak (OIDC) con ruoli applicativi;

- credenziali e URL remoti forniti tramite variabili d'ambiente;
- URL di dettaglio GDACS limitati agli host ufficiali;
- schemi Pydantic che verificano la forma dei dati;
- tool MCP in sola lettura;
- contenuti esterni trattati come dati, mai come istruzioni;
- filtraggio dei dati sensibili nei log applicativi;
- identificativi di richiesta e di conversazione che collegano ogni risposta ai relativi log.

Il prototipo è pensato per un ambiente di sviluppo e ha un limite di sicurezza esplicito: Keycloak protegge l'interfaccia, ma le API del backend non verificano ancora il token né il proprietario dell'artefatto, quindi chi conosce l'identificativo di un artefatto può leggerlo. Prima di un uso multiutente servono la verifica del token sulle API e il controllo del proprietario su conversazioni e artefatti.

I log strutturati registrano servizio, durata, codice di errore e possibilità di retry. Tre endpoint di salute distinguono lo stato del processo, le dipendenze obbligatorie e i controlli remoti.

L'osservabilità copre HTTP e SSE, workflow, nodi, chiamate LLM, tool MCP e accessi al grafo. Prometheus conserva le metriche e Grafana le visualizza; OpenTelemetry esporta i trace a Phoenix. I trace possono includere prompt e output, secondo configurazione. Il Capitolo 7 descrive copertura, interpretazione e limiti di questo livello.

Il principio trasversale dell'architettura riguarda i dati quantitativi: l'LLM seleziona un percorso o propone una vista, mentre acquisizione, filtri, calcoli e validazione sono eseguiti dal codice sui record recuperati.

Capitolo 6

Funzionamento end-to-end, casi d'uso e verifica

Per collegare l'architettura al comportamento osservabile, il capitolo segue alcune richieste dall'ingresso in *Copernicus Data Workspace* fino alla risposta. Gli scenari sono ricavati dal comportamento implementato e i valori citati hanno scopo illustrativo. I risultati di benchmark sono presentati nel Capitolo 8.

6.1 Ingresso della richiesta

Dalla home del workspace l'utente descrive luogo, fenomeno e periodo e avvia una nuova analisi; può anche riprendere un'indagine precedente che conserva dashboard, filtri e contesto. La dashboard invia i messaggi al backend mediante l'endpoint compatibile con le chat completion OpenAI. La richiesta contiene modello, cronologia, opzione di streaming e i metadati da cui si ricava l'identificativo della conversazione. Il backend normalizza tale identificativo e avvia il workflow.

Nel client legacy Open WebUI compaiono anche richieste tecniche generate dall'interfaccia (titolo della chat, suggerimenti), che il sistema riconosce e gestisce separatamente dalle domande geografiche.

Il primo nodo del workflow estrae l'ultimo messaggio dell'utente e lo ripulisce dai marcatori dell'interfaccia. Il Gateway decide quindi come procedere. Da questo punto

sono possibili tre casi:

1. nuova richiesta ammessa: parsing e routing completi;
2. follow-up ammesso: riuso del checkpoint e selezione dei soli nodi necessari;
3. richiesta fuori dominio o classificazione fallita: risposta diretta.

6.2 Scenario 1: analisi multi-sorgente di un pericolo

Si consideri la domanda:

> *"Mostrami i terremoti di magnitudo superiore a 4,5 avvenuti in Italia negli ultimi sette giorni e fammi vedere la copertura satellitare."*

6.2.1 Risoluzione semantica

Il parser produce una richiesta strutturata simile alla seguente:

- categoria: pericolo, sottotipo terremoto;
- luogo: Italia, con bounding box quando il geocoding è disponibile;
- periodo: intervallo assoluto corrispondente agli ultimi sette giorni;
- filtro: magnitudo maggiore di 4,5;
- viste: dati strutturati e mappa.

Il riferimento temporale viene risolto in date precise rispetto all'istante corrente e al fuso orario del backend; il luogo viene trasformato, quando possibile, da etichetta a geometria.

6.2.2 Pianificazione ed esecuzione

Poiché la categoria è un pericolo, il piano comprende Search, GDACS e Copernicus STAC. I tre worker partono in parallelo:

- Search recupera contesto web attuale;

- GDACS chiede al server MCP gli eventi e applica i filtri;
- STAC interroga CDSE per area, periodo e collezioni.

Il filtro GDACS richiede che periodo, magnitudo, tipo e posizione siano soddisfatti dallo stesso record. Se sono presenti coordinate, la verifica spaziale usa la bounding box del luogo oltre al nome del paese. Una bounding box nazionale è un rettangolo e può includere aree confinanti: funziona quindi come filtro geometrico. Per una richiesta cittadina il sistema mantiene l'ambito della città. La risposta riporta gli eventi presenti nel feed interrogato.

Il client STAC invia una ricerca con bounding box e intervallo ISO 8601. Per intervalli o aree estesi può suddividere la richiesta in partizioni temporali e quadranti, con un numero limitato di tentativi. I risultati sono ridotti ai metadati utili e deduplicati per identificativo. Il client legge soltanto la prima pagina di risultati di ciascuna ricerca: per aree o periodi molto ampi i conteggi si riferiscono quindi ai prodotti recuperati, che possono essere meno di quelli presenti nel catalogo.

6.2.3 Sincronizzazione e risposta

LangGraph attende la convergenza dei worker. Il Chart Agent riceve soltanto dataset normalizzati e aggregazioni; il renderer della mappa riceve bounding box e relativi centri. Le acquisizioni STAC alimentano inoltre un artefatto dashboard incorporato nel workspace. Il Conversation Agent compone infine:

1. sintesi testuale;
2. eventuali tabelle;
3. dashboard Copernicus con KPI, grafici, tabella e heatmap;
4. elenco delle fonti consultate.

6.3 Scenario 2: interrogazione di una attivazione CEMS

La domanda:

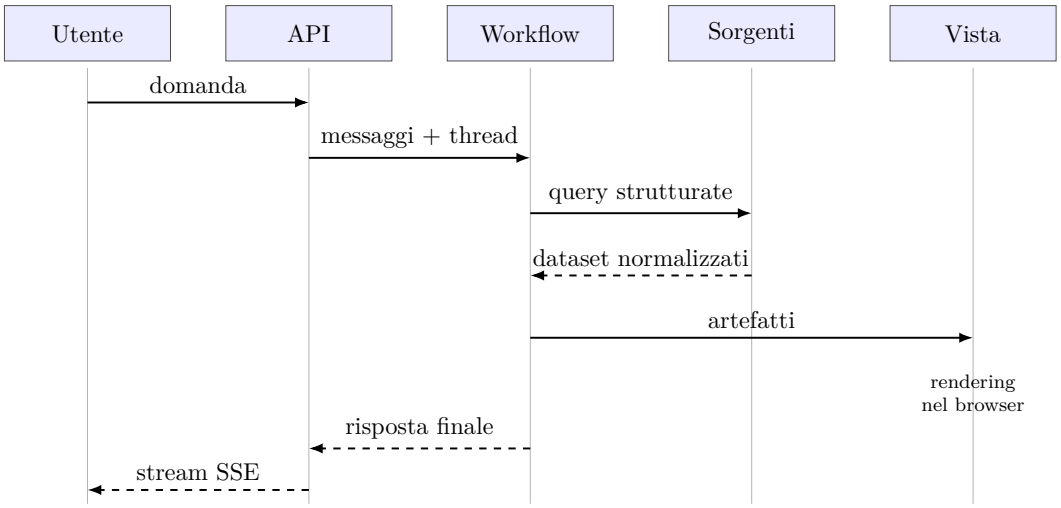


Figura 6.1: Sequenza end-to-end di una richiesta multi-sorgente. Il workflow costruisce l'artefatto dashboard a partire dai record STAC normalizzati; il browser lo carica e lo presenta così com'è.

"Dammi i dettagli dell'attivazione Copernicus EMSR884."

contiene un identificativo univoco. Il ramo CEMS risponde alla domanda diretta; nel prototipo il piano può attivare anche Search e STAC come *arricchimento* per contesto e dashboard. Si tratta di una scelta di prodotto, che aggiunge latenza e dipendenze rispetto alla sola richiesta di dettaglio. Il CEMS Router valida il codice nel formato `EMSR###` o `EMSN###` e sceglie l'endpoint corretto.

Per `EMSR` vengono consultati i dettagli Rapid Mapping; per `EMSN` quelli Risk and Recovery. Il dataset restituito può includere paese, categoria, stato, istante di attivazione, aree di interesse, numero e tipo dei prodotti, URL di download o mappe.

Se una domanda successiva chiede anche le acquisizioni Sentinel nella medesima area, il ramo CEMS viene eseguito prima di STAC e le coordinate dell'attivazione guidano direttamente la ricerca satellitare.

6.4 Scenario 3: follow-up e riuso della memoria

Dopo una prima risposta l'utente può scrivere:

"Ora raggruppali per giorno e mostrami un grafico a barre."

Il messaggio dipende dal contesto. Nell'esempio l'artefatto selezionato è quello delle acquisizioni STAC; se il turno precedente conteneva anche eventi GDACS, il sistema lavora sul dataset attivo nella conversazione; per raggruppare un altro dataset l'utente lo indica esplicitamente. Il Gateway riceve una vista compatta dello stato e riconosce la continuità del dominio. Il Conversation Agent sceglie quindi una delle azioni di follow-up:

- risposta basata sui dati già presenti;
- nuovo grafico o nuova heatmap dello stesso dataset;
- aggiornamento di una singola sorgente;
- aggiornamento di più sorgenti;
- nuova richiesta, se l'utente cambia oggetto.

Nell'esempio il Chart Agent usa il dataset conservato in Redis e aggiorna la visualizzazione, senza ripetere le chiamate GDACS e STAC. Se l'utente opera nella dashboard, il Composer traduce il comando in una mutazione tipizzata e il backend salva una nuova revisione dell'artefatto. Il riuso risparmia chiamate alle sorgenti e il relativo traffico; il guadagno sui tempi richiederebbe una misura controllata. Quando l'utente chiede informazioni correnti, il sistema aggiorna esplicitamente le fonti.

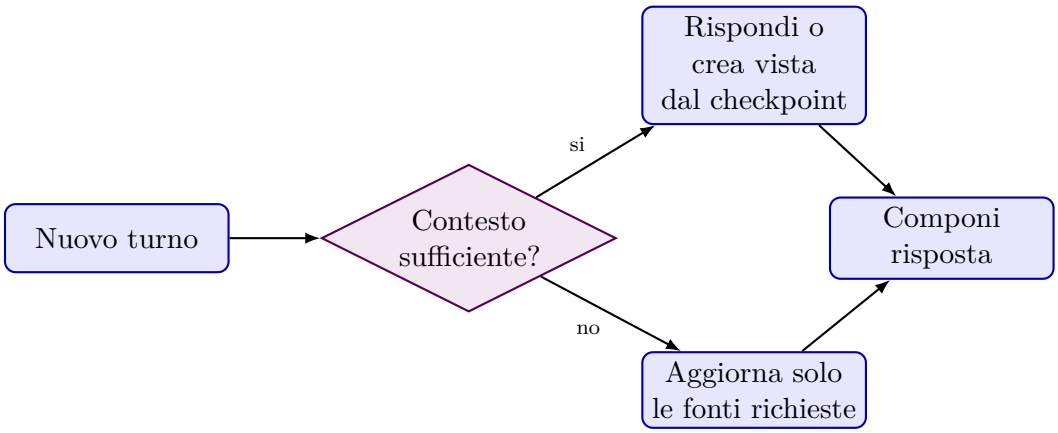


Figura 6.2: Scelta fra riuso della memoria e aggiornamento delle fonti.

6.5 Scenario 4: scoprire un dataset nel grafo

Si consideri la richiesta:

> *"Cerco un dataset orario di temperatura superficiale del Mar Nero: quale variabile contiene?"*

Il Gateway riconosce un compito di Discovery. Il nodo d'intento rappresenta il tema temperatura superficiale, l'area Mar Nero, la risoluzione oraria e il tipo dataset con variabile. Il servizio recupera candidati dal sottografo Marine, ne verifica la compatibilità e applica il ranking deterministico descritto nella Sezione 5.5.5.

Lo snapshot di riferimento del benchmark, salvato il 13 ottobre 2026 e riportato nella guida [29], associa questa ricerca al dataset `cmems_obs-sst_blk_phy-sst_nrt_diurnal-oi-0.0625deg_PT1H-m`, dal titolo *Black Sea Diurnal Subskin SST Analysis, L4, 1/16deg hourly*, con variabile `analysed_sst`. La bounding box registrata è [26,375; 38,75; 42,375; 48,8125] nell'ordine ovest, sud, est, nord. Si tratta di metadati del catalogo, utili a individuare la fonte.

La risposta presenta dataset, variabile e provenienza, e l'artefatto permette di consultare le risorse selezionate. La scoperta della fonte scientifica resta così distinta dalla ricerca di scene satellitari; l'analisi dei valori della variabile è fuori dal perimetro del prototipo.

Se un turno successivo chiede le acquisizioni relative alla stessa area, il passaggio al percorso operativo conserva il riferimento geografico. Per verificarlo occorre confrontare intento, query effettiva, artefatto salvato e viste mostrate nel browser; questo controllo conversazionale è distinto dai casi a thread nuovo del benchmark.

6.6 Dashboard Copernicus

La dashboard analitica è il canale principale per le visualizzazioni delle acquisizioni satellitari. Nel ramo operativo viene aperta nel workspace come artefatto a sé (vista "Analisi dati Copernicus"), con navigazione dedicata a risultato, grafici, mappa, dati e attività, più una chat laterale sul risultato già caricato. Grafici, KPI, tabella e heatmap usano esclusivamente record STAC Copernicus normalizzati; Search, GDACS e CEMS

compaiono nella risposta testuale o in sezioni separate. In questo modo, per esempio, il numero di eventi GDACS e il numero di prodotti satellitari restano separati.

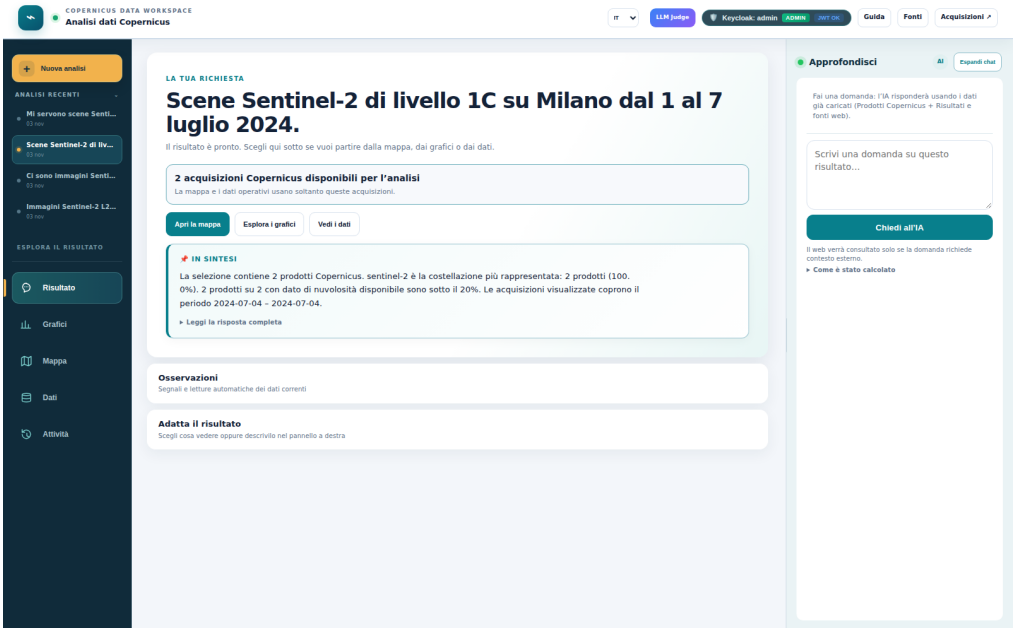


Figura 6.3: Sessione operativa ripresa dal workspace: sintesi delle acquisizioni (conteggio di prodotti/item STAC recuperati), navigazione Risultato/Grafici/Mappa/Dati e chat laterale sul risultato già caricato.

Il ramo Discovery presenta invece un artefatto del catalogo, con schede e metadati delle risorse pertinenti. Le due viste rispondono a compiti diversi e indicano ciascuna la propria provenienza.

6.6.1 Dati, provenienza e composizione

Il Chart Agent costruisce una specifica JSON con valori derivati dal dataset: etichette, serie, tipo di grafico, campo e misura di origine, filtri, numero di record e identificativi dei prodotti. Il modello può proporre il tipo di vista o una nuova aggregazione, mentre conteggi, percentuali, medie, variazioni e confronti sono sempre calcolati dal backend. La sezione "Come è stato calcolato" e la tabella dei dati di ciascun grafico rendono ispezionabile questa provenienza; i dati possono inoltre essere esportati in CSV.

La composizione iniziale segue gli output richiesti: una domanda come "solo la heatmap" apre soltanto la mappa, mentre una richiesta generale include KPI, grafici,

tabella, sintesi e mappa. In assenza di acquisizioni l'artefatto mostra uno stato vuoto esplicito.

6.6.2 Grafici leggibili e senza sovrapposizioni

Le regole di rendering descritte nella Sezione 5.10.1 vengono applicate alla specifica generata prima che la dashboard sia mostrata. Il Dashboard Validator riallinea etichette, valori e serie e intercetta valori non numerici; ECharts gestisce poi dimensioni, etichette e scorrimento nel browser. La verifica dei dati resta così separata dalle correzioni puramente grafiche.

6.6.3 Interazione, revisioni e chat contestuale

Nel workspace i controlli visivi permettono di scegliere barre, linee o torta, ordinamento, Top N e visibilità dei grafici. Un click su categoria o cella può attivare un cross-filter: il backend applica il filtro ai record e ricalcola simultaneamente KPI, grafici, tabella e heatmap. I filtri attivi sono mostrati e possono essere azzerati con un solo comando.

Ogni modifica passa attraverso azioni tipizzate, validate e salvate. Sono disponibili annullamento, ripristino e confronto tra revisioni. La chat contestuale sul risultato ha un thread indipendente per grafico, tabella, heatmap o dashboard: risponde sul sottoinsieme corrente e trasforma istruzioni come "filtra solo Sentinel-2A" o "cambia questo grafico in linee" in modifiche dell'artefatto. La ricerca web interviene solo quando serve un contesto esterno ai dati visualizzati. La home del workspace conserva inoltre l'elenco delle indagini recenti, così da riprendere una sessione senza ripetere la domanda iniziale.

6.6.4 Heatmap Copernicus

Leaflet rappresenta una heatmap della densità dei centri delle bounding box degli item recuperati e la collega a una vista con i rettangoli delle bounding box e i metadati STAC. La heatmap descrive quindi dove si concentrano le acquisizioni, e non la copertura areale o l'intensità del fenomeno. Se un cross-filter è attivo, la mappa usa lo stesso sottoinsieme di record degli altri componenti.

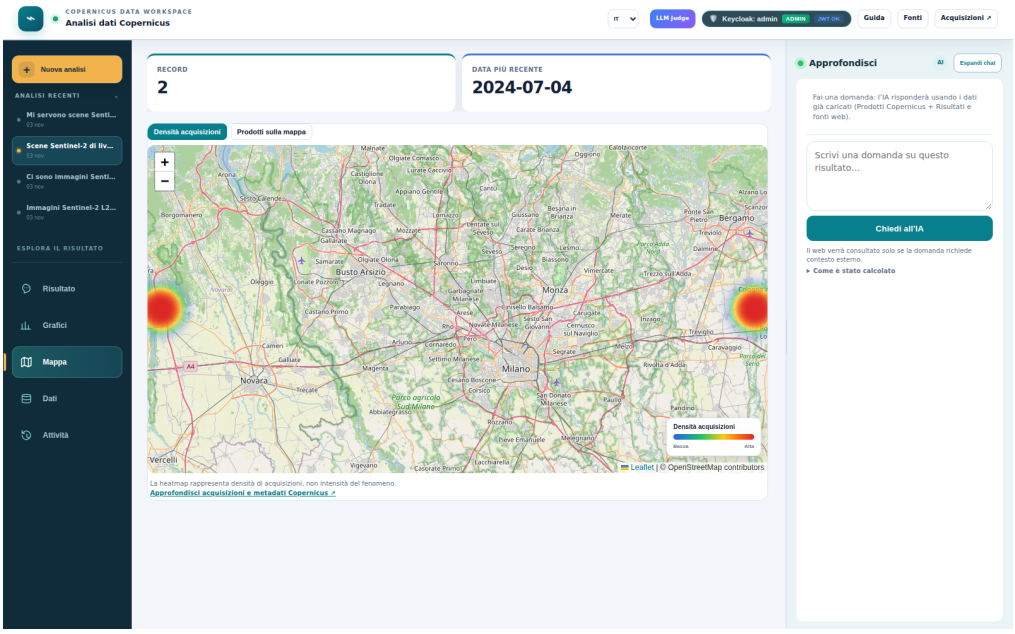


Figura 6.4: Vista *Mappa*: densità dei centri delle bounding box degli item STAC recuperati e chat laterale sul risultato.

6.7 Streaming della risposta

Durante l'esecuzione il backend invia aggiornamenti sullo stato delle fasi (interpretazione, routing, recupero delle fonti, composizione). FastAPI traduce gli eventi LangGraph in Server-Sent Events nel formato delle chat completion OpenAI; il workspace li mostra come avanzamento della richiesta.

Il testo generato dal Conversation Agent viene rifinito prima della consegna: tabelle Search/GDACS, fonti e dashboard Copernicus sono aggiunte in un ordine stabile. Lo stream pubblica la versione finale completa di artefatto e si chiude con un marcatore di fine.

6.8 Gestione degli errori

6.8.1 Errore di una sorgente

Ogni chiamata a una sorgente segue un contratto di errore comune. Se GDACS fallisce mentre Search e STAC terminano correttamente, il grafo conserva i risultati disponibili

e il Conversation Agent produce una risposta parziale che indica con precisione la fonte mancante.

Anche all'interno di STAC una ricerca partizionata può fallire soltanto per alcune aree o finestre temporali. I prodotti ottenuti restano utilizzabili e la copertura incompleta viene segnalata nella risposta e nell'artefatto.

Gli errori sono classificati in configurazione, autenticazione, rete, timeout, risposta non valida e persistenza. A ciascuno sono associati:

- codice pubblico stabile;
- identificativo diagnostico univoco;
- servizio coinvolto;
- indicazione sulla possibilità di ripetere l'operazione;
- dettaglio tecnico destinato soltanto ai log.

6.8.2 Timeout e retry

I retry sono riservati agli errori presumibilmente transitori, come timeout, problemi di rete, HTTP 429 e HTTP 5xx. Anche il modello remoto dispone di timeout, backoff e limite di concorrenza, così che i rami paralleli restino entro la capacità del runner Ollama.

6.8.3 Errore della memoria

Redis è una dipendenza obbligatoria: senza memoria un follow-up potrebbe sembrare valido pur avendo perso il contesto precedente. Per questo il servizio si dichiara pronto solo quando memoria e grafo sono disponibili.

6.9 Matrice dei percorsi e delle evidenze

La Tabella 6.1 collega percorsi tipici alle fonti previste. La Tabella 6.2 indica, per le funzionalità descritte nel capitolo, il tipo di evidenza disponibile: il benchmark del

Capitolo 8 copre i casi di ricerca nel grafo e di acquisizioni, mentre GDACS, CEMS, follow-up, autenticazione e ripresa della sessione sono documentati da test e scenari.

Richiesta	Rami principali	Risultato atteso
Clima in Sicilia	Piano operativo	Contesto web e acquisizioni secondo il piano, senza stime climatiche dai raster.
Dataset orario SST del Mar Nero	Discovery su NebulaGraph	Risorsa Marine, variabile e metadati di compatibilità.
Siccità in Sicilia e acquisizioni	Discovery + operativo	Risorse Drought pertinenti e acquisizioni STAC nella stessa sessione.
Terremoti in Indonesia	Search + GDACS + STAC	Eventi filtrati, contesto web e copertura delle acquisizioni.
Dettagli EMSR884	CEMS + Search/STAC	Attivazione e prodotti; arricchimento dashboard se abilitato.
Grafico dei risultati	Chart su contesto	Nuova vista sui dati già recuperati.
Aggiornamento GDACS	Refresh GDACS	Sostituzione del solo dataset eventi.
Ripresa sessione dal workspace	Checkpoint Redis	Ricarico di filtri, artefatti e cronologia utili.
Domanda fuori dominio	Gateway	Risposta diretta, senza chiamate esterne.

Tabella 6.1: Esempi di instradamento del workflow e di ripresa sessione.

Funzionalità	Test mock	Integrazione	Benchmark	Browser
Discovery (grafo)	Parziale	Harvest/repo	Sì (51)	Artefatto
STAC live	Query/filtri	Client CDSE	Sì (54)	Dashboard
GDACS / CEMS / web	Normalizz.	MCP/HTTP	No	Scenario
Follow-up / revisioni	Workflow	API mutazioni	No	Scenario
Keycloak / sessioni	—	OIDC	No	Accesso
Osservabilità	Metriche	Overlay	No	Grafana

Tabella 6.2: Funzionalità descritte e tipo di evidenza disponibile. "Benchmark" si riferisce al protocollo 6.0 del Capitolo 8; "Browser" indica il tipo di schermata illustrativa disponibile.

6.10 Strategia di verifica

La verifica è organizzata per livelli.

6.10.1 Test unitari

I test unitari controllano normalizzazione dei feed GDACS, sicurezza degli URL, query STAC, generazione di grafici, contratti di errore e trasformazioni temporali. Per la dashboard verificano inoltre la correzione di valori e serie anomali, il ricalcolo del cross-filter su grafici, KPI, tabella e heatmap, e la provenienza delle metriche generate. Si tratta di prove deterministiche, che confrontano input e output noti indipendentemente dal modello linguistico.

6.10.2 Test del workflow

I test del grafo sostituiscono le dipendenze remote con simulazioni (mock) e verificano:

- selezione delle sorgenti e parallelismo condizionale;
- convergenza dei risultati nel Conversation Agent;
- separazione della memoria tra conversazioni;
- chiarimenti su richieste incomplete;
- follow-up dal contesto e aggiornamenti selettivi;
- propagazione della risposta finale nello streaming.

6.10.3 Test delle integrazioni

I client MCP e Copernicus sono verificati rispetto a timeout, errori HTTP, risposte malformate e tentativi limitati. I test della dashboard coprono le chiamate per chat, modifiche, annullamento, versioni e confronto, oltre ai controlli dell'interfaccia.

Al 15 ottobre 2026 risultavano superati, fra gli altri, 103 test del modulo di valutazione e 32 test del pacchetto di osservabilità [29]. Poiché molte prove usano dipendenze simulate, esse verificano la logica del sistema e lasciano alle sorgenti live il compito

di confermarne il comportamento reale. Il benchmark del Capitolo 8 aggiunge una misura del workflow su domande di riferimento; la verifica dell'interfaccia renderizzata costituisce un livello separato.

6.11 Limiti attuali

L'architettura riduce alcuni rischi, mentre altri dipendono dalle fonti e dai modelli:

- GDACS contiene stime che possono essere aggiornate;
- una acquisizione STAC disponibile può essere poco adatta a valutare un danno;
- il geocoding può essere ambiguo;
- l'LLM può scegliere una formulazione imprecisa, anche con dati strutturati;
- ricerca web e servizi remoti dipendono dalla rete.

Il protocollo implementato aggiunge domande con riferimenti, controlli di instradamento e filtri, voti di qualità e statistiche per categoria; le metriche operative permettono di localizzare latenze e guasti. Queste evidenze sostengono conclusioni circoscritte al benchmark e ai percorsi osservati. Il vantaggio quantitativo del parallelismo e del riuso della memoria richiede confronti controllati, e la qualità in un impiego reale va valutata con prove conversazionali che controllino anche il risultato visualizzato.

Capitolo 7

Osservabilità del workflow e diagnosi

Una risposta corretta, un'esecuzione priva di eccezioni e un servizio raggiungibile sono proprietà distinte. Il livello di osservabilità permette di esaminare le ultime due e di ricostruire il percorso che ha prodotto la prima; la valutazione semantica è invece affidata ai riferimenti e ai controlli del Capitolo 8. Questo capitolo descrive la strumentazione LLMOps del prototipo, basata su Prometheus, Grafana e OpenTelemetry/Phoenix, e il modo in cui i suoi dati vanno letti [29].

7.1 Log, metriche e trace

I log descrivono eventi puntuali, come un timeout o un nuovo tentativo. Le metriche riassumono quantità numeriche nel tempo, come il numero di esecuzioni e la distribuzione delle durate. Un trace collega le operazioni di una singola richiesta: ciascuno span rappresenta un intervallo con i suoi attributi, l'eventuale errore e la relazione con le operazioni che lo contengono [27].

Le tre rappresentazioni rispondono a domande complementari. Un aumento delle durate segnala un gruppo di richieste lente, il trace individua il passaggio coinvolto e il log precisa l'errore. La correttezza del contenuto resta una questione separata: una ricerca valida sulla città sbagliata termina tecnicamente con successo.

7.2 Architettura della telemetria

Prometheus interroga il backend ogni 15 secondi e conserva le serie temporali delle metriche; Grafana le presenta in dashboard. OpenTelemetry invia invece i trace a Phoenix, che permette di esplorare il percorso di ogni richiesta e di raggruppare le richieste di una stessa conversazione [2]. I due flussi sono rappresentati nella Figura 7.1.

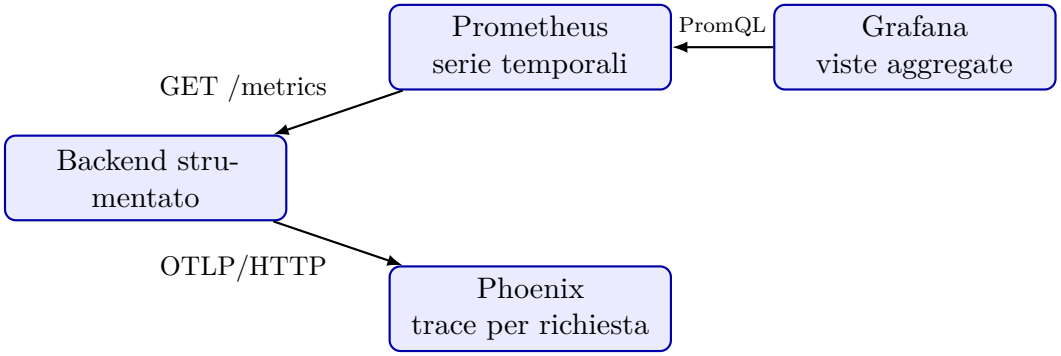


Figura 7.1: Due flussi della telemetria: raccolta delle metriche in modalità pull ed esportazione dei trace in modalità push. Le frecce indicano l'iniziatore della richiesta o dell'invio.

L'osservabilità si attiva come estensione opzionale del deployment e strumenta la stessa applicazione, senza modificarne il codice. I dati sono conservati per 15 giorni. I risultati del Judge restano nei propri file e nella propria interfaccia, separati dalla telemetria operativa.

7.3 Che cosa viene misurato

La strumentazione avvolge le operazioni principali del backend, ne misura la durata e ne registra l'esito. Copre quattro livelli.

- **Richieste HTTP e chat.** Per ogni richiesta sono registrati stato e durata. Per la chat in streaming sono distinti il primo evento, il primo contenuto testuale e la conclusione, perché un aggiornamento di stato precede di solito la risposta vera e propria. L'esito finale distingue completamento, errore, interruzione e disconnessione del client.

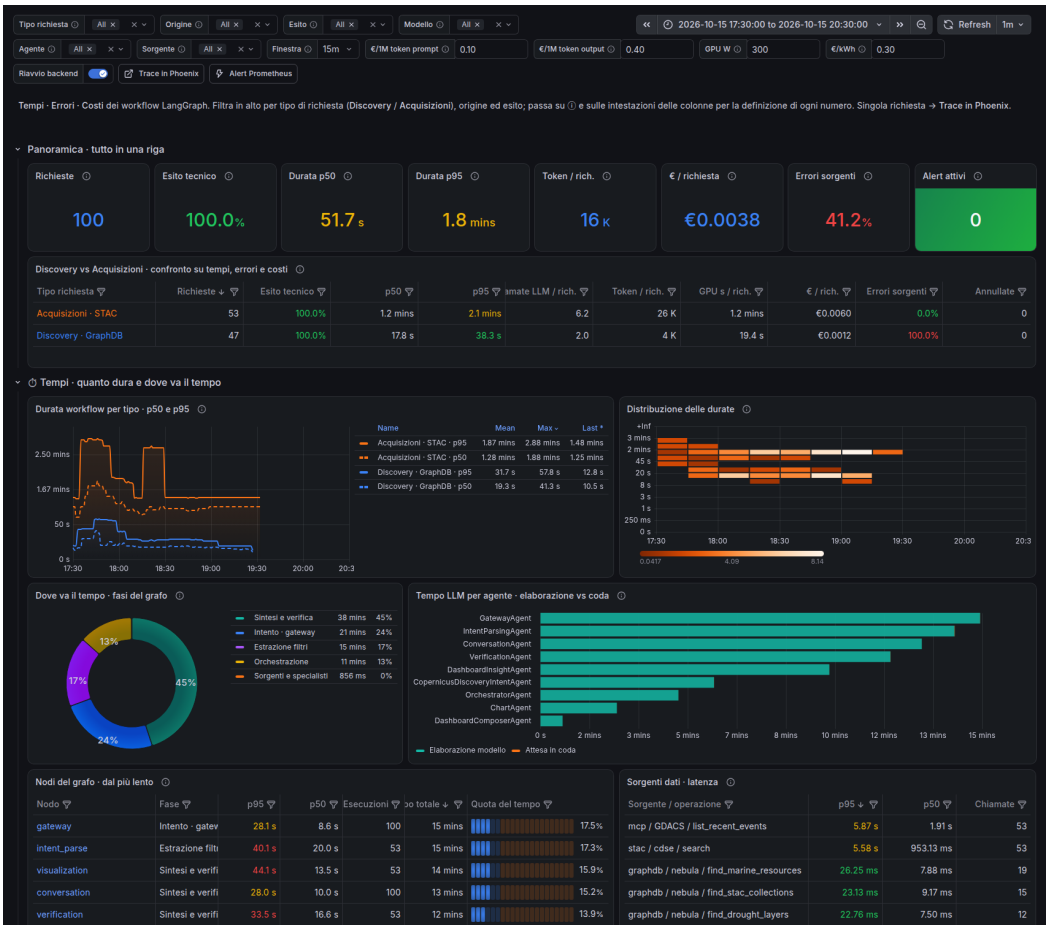


Figura 7.2: Dashboard Grafana *Copernicus · LLMOps*: panoramica di richieste, esiti, durate e confronto Discovery/acquisizioni, con collegamenti a Phoenix e agli alert Prometheus.

- **Workflow e nodi.** Ogni esecuzione del workflow e ogni nodo del grafo producono uno span con durata, esito e stadio logico (intento, estrazione, orchestrazione, specialista, sintesi).
- **Strumenti e catalogo.** Sono tracciate le chiamate agli strumenti MCP e le query al catalogo a grafo, con il numero di risultati.
- **Chiamate al modello.** Per ciascuna invocazione sono separati il tempo d'attesa in coda, il tempo di inferenza e gli eventuali nuovi tentativi; sono registrati anche i token in ingresso e in uscita forniti dal server del modello [23], da cui si ricava la

velocità di generazione in token al secondo.

Il conteggio dei token rappresenta il consumo registrato dalle chiamate osservate, utile per confrontare agenti e periodi; il Judge usa un canale proprio ed è escluso da questo conteggio. Le durate degli span sono annidate: il tempo di un nodo è già compreso in quello del workflow che lo contiene.

7.4 Come leggere le metriche

Prometheus usa tre tipi di misura: i contatori, che crescono nel tempo (per esempio il numero di richieste); i gauge, che indicano un valore corrente (per esempio le chiamate in corso); gli istogrammi, che raggruppano le durate in fasce e permettono di stimarne la distribuzione [30]. I valori per periodo mostrati in dashboard sono stime ricavate da queste serie.

Per le durate la dashboard riporta due percentili. Il p50, o mediana, è la durata entro cui si conclude metà delle richieste. Il p95 è la durata entro cui si conclude il 95% delle richieste: descrive le richieste più lente senza farsi dominare da un singolo caso estremo. È quindi un indicatore di reattività, distinto sia dalla durata massima sia dalla qualità della risposta, e diventa affidabile quando il numero di campioni è sufficiente.

La dashboard calcola due tassi di successo su popolazioni diverse:

$$r_{\text{workflow}} = \frac{N_{\text{ok}}}{N_{\text{ok}} + N_{\text{error}}}, \qquad r_{\text{chat}} = \frac{N_{\text{completed}}}{N_{\text{completed}} + N_{\text{error}} + N_{\text{interrupted}}}.$$

Il primo esclude le esecuzioni annullate, il secondo le disconnessioni del client; entrambi vanno letti insieme alle tabelle degli esiti. In assenza di traffico il rapporto resta vuoto, condizione distinta da un tasso pari a zero.

7.5 Dashboard Grafana e alert

La dashboard è generata da uno script e caricata automaticamente all'avvio [13]: la versione nel repository è quella di riferimento. È organizzata in sezioni tematiche, riassunte nella Tabella 7.1, e include una guida espandibile che spiega unità di misura, percentili, token ed esiti.

Sezione	Domanda a cui risponde
Panoramica	Il sistema sta raccogliendo dati? Quante richieste riescono e in quanto tempo?
Diagnosi	Da dove vengono gli errori? Quali nodi e servizi esterni sono più lenti?
LLM	Quante chiamate al modello sono attive o in coda? Quanti tentativi e quanti token per agente?
Chat	Quanto tempo passa prima della prima risposta? Come si concludono le conversazioni?
HTTP e stadi	Quanto traffico riceve ogni servizio e quanti errori del server produce?

Tabella 7.1: Sezioni principali della dashboard, affiancate dalla guida alle metriche.

I filtri permettono di restringere le viste per tipo di richiesta, esito, modello o agente (Figura 7.3); la sezione dedicata al modello è mostrata nella Figura 7.4. Prima di leggere uno storico conviene controllare l'indicatore di raccolta, che segnala se i dati sono aggiornati.

Le regole di alert coprono backend irraggiungibile, errori HTTP, stream falliti, prima risposta lenta, coda del modello e guasti degli strumenti. Un alert scatta quando la condizione persiste per un intervallo prefissato. Gli alert sono visibili in Prometheus e in Grafana; nel prototipo manca un canale di notifica verso persone o sistemi esterni.

7.6 Diagnosi e copertura

Quando una richiesta è lenta, il percorso di analisi parte dalla panoramica (raccolta attiva, numero di campioni) e scende verso workflow, nodi, coda del modello, tentativi e servizi esterni. Una coda lunga con inferenza stabile indica attesa per il modello; un nodo lento con chiamate al modello brevi orienta verso altri passaggi. Il trace della richiesta conferma l'ipotesi mostrando dove si concentra il tempo, e i log precisano gli errori.

Quando una risposta è sbagliata, il confronto riguarda domanda, luogo interpretato, query, record restituiti e artefatto finale. Qui gli indicatori tecnici possono risultare tutti positivi: la verifica del contenuto spetta al benchmark e, per tabella e mappa, a

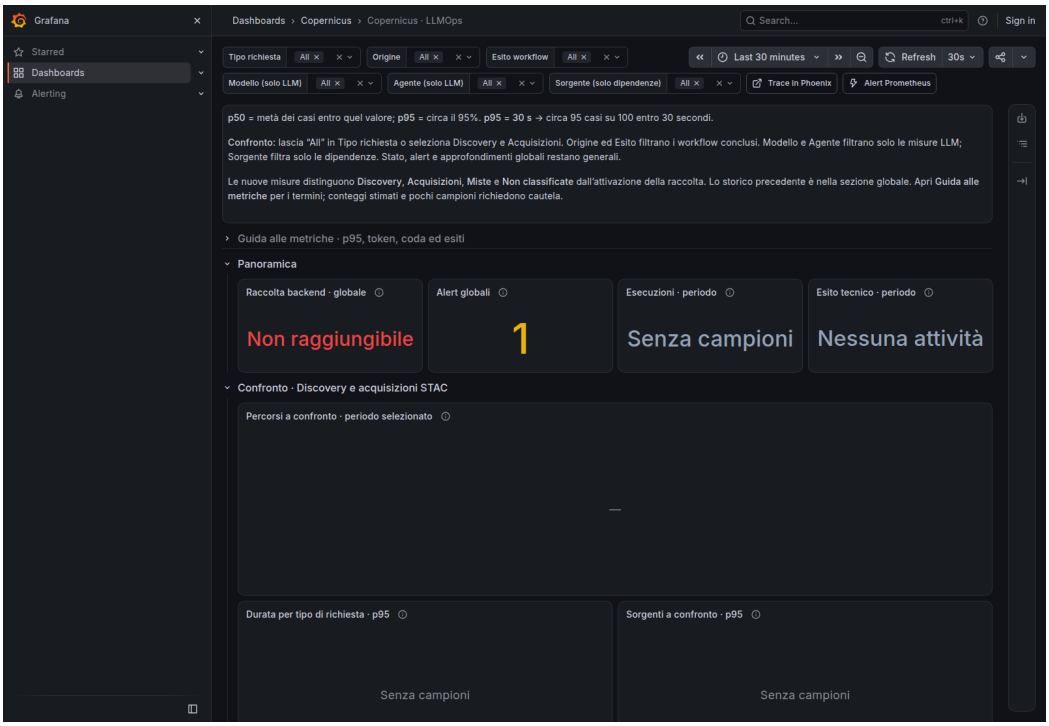


Figura 7.3: Filtri della dashboard LLMOps (tipo richiesta, origine, esito, modello, agente, sorgente) e riga di panoramica con stato della raccolta e alert.

una prova dell'interfaccia.

La copertura riguarda tempi, esiti e consumo di token. Restano fuori l'uso della GPU, le risorse dei singoli container e l'accuratezza semantica; alcune dipendenze, come il catalogo STAC, la ricerca web e il Judge, compaiono solo attraverso il tempo del nodo che le invoca. I trace possono inoltre registrare prompt e risposte: la cattura del contenuto è configurabile e limitata in lunghezza, ma va considerata un dato potenzialmente sensibile.

La ricognizione del 15 ottobre 2026 illustra questi aspetti: Grafana mostrava ancora i dati storici mentre Prometheus segnalava il backend irraggiungibile, con l'alert corrispondente attivo [29]. La presenza di grafici, la salute corrente del servizio e la correttezza dei risultati vanno quindi verificate separatamente.

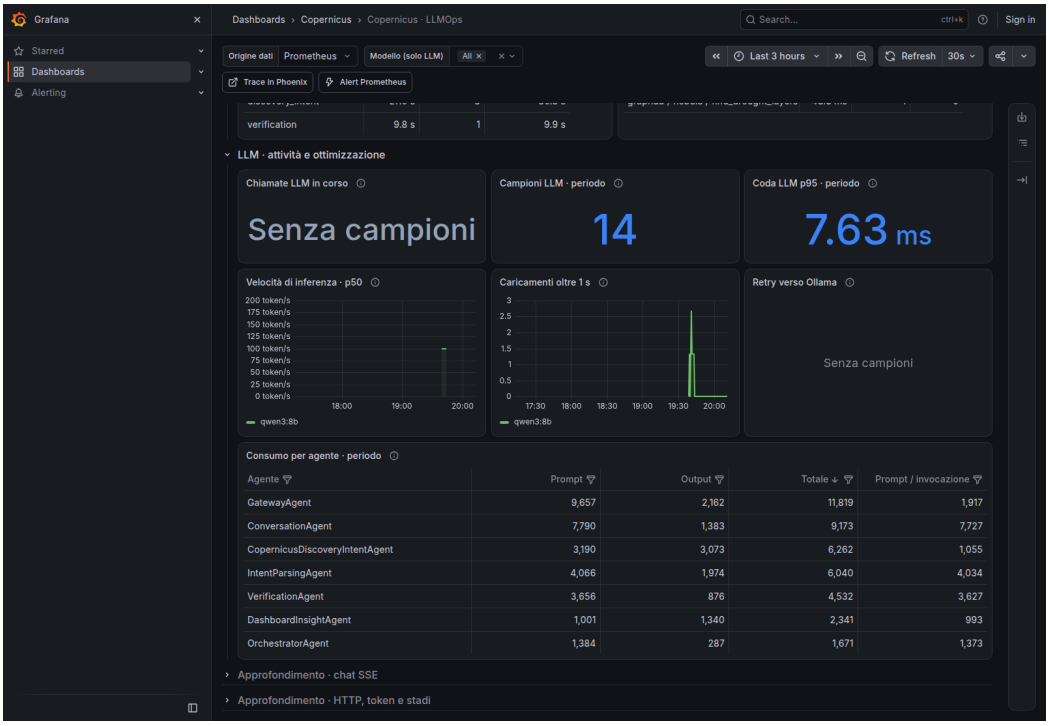


Figura 7.4: Sezione LLM della stessa dashboard: coda, retry, token per agente e salute delle invocazioni al modello.

Capitolo 8

Valutazione con LLM-as-a-Judge

Valutare un agente significa controllare anche che cosa ha cercato, oltre alla forma delle sue risposte: un workflow può rispettare gli schemi e produrre un testo scorrevole riferito al luogo o al periodo sbagliato. Il progetto combina quindi un benchmark ancorato a evidenze, controlli deterministici e un LLM-as-a-Judge. Questo capitolo descrive il **protocollo 6.0** e ne riporta i risultati [28]. In chiusura si richiamano, come riferimento storico, le run con protocollo 4.0 [29], che seguono regole e dataset diversi.

8.1 Oggetto e perimetro della valutazione

Il sistema sotto esame è il `WorkflowAgent` nel suo insieme: instradamento, interpretazione, recupero e sintesi. Ogni caso del benchmark gli sottopone la domanda originale; il Judge valuta poi la risposta e le evidenze osservate. Nella configurazione documentata l'agente usa `qwen3:8b` e il Judge `gemma3:27b`: impiegare una famiglia di modelli diversa riduce la tendenza di un modello a premiare risposte simili alle proprie.

Il Judge è un modello linguistico generico, interrogato con rubriche esplicite e con un formato di risposta strutturato. Opera a posteriori su casi di cui è nota la risposta attesa, ed è quindi distinto dal nodo Verification, che controlla l'agente durante il singolo turno.

La letteratura documenta che i giudizi di un LLM possono risentire di posizione, lunghezza della risposta e affinità fra modelli [41]. Per questo il protocollo legge i voti

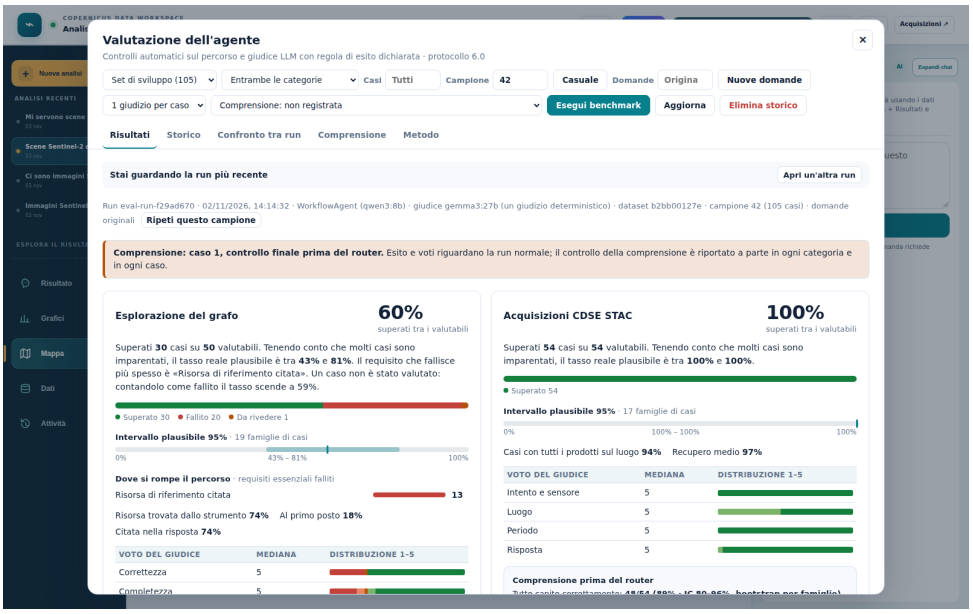


Figura 8.1: Schermata illustrativa del pannello di valutazione: dataset, Judge, controlli e sintesi per categoria. I conteggi ufficiali sono quelli della Tabella 8.2.

del Judge insieme ai controlli deterministici e alle misure di incertezza descritte più avanti.

8.2 Dataset ed evidenze di riferimento

8.2.1 Composizione dei casi

Il dataset corrente contiene **105 casi**: 51 sul catalogo a grafo e 54 sulle acquisizioni satellitari, raggruppati rispettivamente in 19 e 17 famiglie di scenari. Ogni caso conserva domanda, risposta attesa, strumenti attesi, evidenze e famiglia di appartenenza.

I casi del grafo riguardano risorse Marine, Drought e CDSE/STAC catalogate in NebulaGraph. Le acquisizioni usano le collezioni Sentinel-2 L1C/L2A e Sentinel-1 GRD/SLC; alcuni casi prevedono un risultato vuoto, perché anche cercare correttamente e dichiarare l'assenza di prodotti è un comportamento corretto. Un set di domande perturbate è riservato a future prove di generalizzazione.

Le famiglie raccolgono parafrasi dello stesso scenario: ampliano la varietà linguistica, ma domande sullo stesso luogo tendono a condividere gli stessi errori. Per questo i

risultati vanno letti tenendo conto sia del numero di casi sia del numero di famiglie.

8.2.2 Costruzione della ground truth

Per il grafo, ogni vincolo citato nella domanda (titolo, tipo, area, risoluzione, variabili) viene verificato sui metadati della risorsa; solo i casi pienamente supportati diventano riferimenti validi.

Per le acquisizioni, il riferimento comprende la ricerca attesa (area, periodo, collezione) e i prodotti osservati. Un sottoinsieme di questi, i prodotti che coprono il centro del luogo richiesto (*core_ids*), serve da insieme di riferimento per misurare il recupero.

I riferimenti sono istantanee dei metadati del catalogo. Poiché il catalogo remoto evolve, una differenza negli identificativi va interpretata alla luce della ricerca effettuata e dei prodotti restituiti.

8.3 Esecuzione del vero workflow

Ogni caso esegue l'agente reale, con la sola domanda e una conversazione nuova. L'agente deve ricavare da sé area, sensore e periodo: in questo modo il benchmark misura anche comprensione e instradamento, oltre al client di ricerca. Per le acquisizioni viene registrata la traccia completa: interpretazione, ricerca effettivamente eseguita e prodotti restituiti.

Per ogni caso la sequenza comprende esecuzione dell'agente, controlli deterministici, giudizio, verifica della coerenza dei voti, decisione finale e salvataggio della run. Gli strumenti effettivamente usati sono ricavati da ciò che l'agente ha prodotto e confrontati con quelli attesi dal caso, così che un errore di instradamento emerga in modo esplicito. Ogni caso ha un tempo massimo configurabile, di 600 secondi per default.

8.4 Controlli deterministici e rubriche ordinali

8.4.1 Controlli sui fatti osservati

Ogni controllo produce un esito (vero, falso o non applicabile) e una gravità: `critical`, `major`, `minor` o `info`. Un fallimento critico determina l'esito negativo del caso; gli altri

vengono riportati nel risultato e mostrati al Judge come segnali di qualità.

Per il grafo si verifica che l'agente segua il percorso Discovery e si misurano recupero, posizione e citazione della risorsa di riferimento, oltre alla presenza dei campi richiesti. Quando la domanda ha un unico target, citarlo è un requisito critico.

Per le acquisizioni sono critici, fra gli altri: il percorso STAC; luogo, periodo e sensore della ricerca; i filtri richiesti dall'utente, applicati senza aggiungerne altri; una risposta ottenuta in tempo reale da CDSE; l'uso esclusivo di identificativi realmente restituiti. Controlli di gravità `major` riguardano, per esempio, la quota di prodotti che cadono sul luogo e la coerenza della sintesi con i prodotti selezionati.

Quando una ricerca corretta incontra un guasto della sorgente esterna, il caso è classificato come non valutabile; un luogo o un periodo errati restano invece un fallimento dell'agente.

8.4.2 Dimensioni e soglie

I voti sono interi da 1 a 5, con una descrizione esplicita di ogni livello. Le rubriche del protocollo 6.0 differiscono per categoria, come mostra la Tabella 8.1.

Categoria	Dimensione	Oggetto del giudizio	Soglia
Grafo	correctness	Correttezza della risorsa e dei vincoli.	4
	completeness	Presenza degli elementi richiesti.	4
	relevance	Focalizzazione e pertinenza.	3
	faithfulness	Aderenza all'output dello strumento.	4
Acquisizioni	intent	Sensore/prodotto e filtri richiesti.	4
	place	Luogo effettivamente interrogato.	4
	period	Intervallo effettivamente interrogato.	4
	response	Coerenza della sintesi con i prodotti selezionati.	4

Tabella 8.1: Dimensioni ordinali e soglie minime del protocollo 6.0. I controlli critici si applicano indipendentemente dai voti.

Per le acquisizioni ogni errore sostanziale è attribuito a una sola dimensione (per esempio area troppo ampia → `place`; filtri sbagliati → `intent`). Sul grafo, `faithfulness` misura l'aderenza della risposta a ciò che lo strumento ha restituito: dichiarare "nessun risultato" quando lo strumento è vuoto è fedele, anche se scorretto rispetto al riferimento.

Un caso supera la valutazione solo se *ogni* dimensione raggiunge la propria soglia; la media dei voti ha valore descrittivo. Una sequenza 5, 5, 5, 1, per esempio, ha media 4 ma fallisce.

8.4.3 Prompt e coerenza dei voti

Il Judge riceve domanda, riferimento, risposta, traccia e risultati dei controlli. Opera con temperatura zero, seed fisso, contesto di 16.384 token e risposta in formato strutturato. Se i voti contraddicono i fatti verificati, il giudizio viene ripetuto; una contraddizione persistente porta il caso nello stato `review`.

8.5 Stati finali e denominatori

Ogni caso termina in uno di cinque stati:

`pass` controlli critici rispettati e soglie superate;

`fail` fallimento critico o almeno un voto insufficiente;

`review` incoerenza residua dei voti, da esaminare a mano;

`not_evaluable` evidenza inutilizzabile, guasto esterno su richiesta corretta o valutatore inutilizzabile;

`error` errore, timeout o assenza di risposta dell'agente.

Il protocollo calcola due tassi di successo. Il *primario* considera solo i casi con esito netto (pass o fail); il *rigoroso* mette al denominatore tutti i casi, contando come insuccesso ogni stato diverso da pass. Indicando con P, F, R, U, E i conteggi dei cinque stati:

$$p_{\text{primario}} = \frac{P}{P+F}, \qquad p_{\text{rigoroso}} = \frac{P}{P+F+R+U+E}. \tag{8.1}$$

Se una run si interrompe, i casi completati vengono comunque salvati e la run è marcata come parziale. Grafo e acquisizioni sono sempre riportati separatamente.

8.6 Incertezza e misure di recupero

8.6.1 Intervallo di Wilson

Su un numero limitato di casi, una percentuale osservata è una stima. Per indicarne l'incertezza il protocollo usa l'intervallo di Wilson al 95%, adatto anche a proporzioni vicine a 0 o a 1. Con $n = P + F > 0$ casi valutabili, proporzione osservata $\hat{p} = P/n$ e $z \simeq 1{,}96$:

$$\frac{\hat{p} + z^2/(2n) \;\pm\; z\sqrt{\hat{p}(1-\hat{p})/n + z^2/(4n^2)}}{1 + z^2/n}. \tag{8.2}$$

La procedura è quella delle librerie statistiche di riferimento [38]. Wilson tratta ogni caso come un'osservazione indipendente.

8.6.2 Bootstrap per famiglie e voti ordinali

Per tener conto delle parafrasi, il bootstrap per famiglie ricampiona interi scenari anziché singole domande, preservando la dipendenza al loro interno. I due intervalli rispondono quindi a ipotesi diverse e vengono riportati affiancati. Quando tutti i casi passano, il bootstrap si riduce a $[100\%, 100\%]$, perché il campione contiene solo successi; in quel caso l'informazione utile sull'incertezza viene da Wilson e dalla numerosità.

I voti da 1 a 5 sono ordinali: per ogni dimensione il sistema riporta distribuzione, mediana e quartili.

8.6.3 Indicatori indipendenti dai voti

Accanto ai voti, il protocollo misura direttamente la qualità del recupero. Per il grafo conta se la risorsa di riferimento compare fra i risultati (*hit*), se compare per prima (*hit@1*) e quanto in alto si trova in media (*MRR*, mean reciprocal rank). Indicando con r_i la posizione del riferimento nel caso i su N casi, con $1/r_i = 0$ quando il riferimento manca e con $\mathbf{1}[\cdot]$ la funzione indicatrice:

$$\text{hit} = \frac{1}{N}\sum_i \mathbf{1}[r_i < \infty], \qquad \text{hit@1} = \frac{1}{N}\sum_i \mathbf{1}[r_i = 1], \qquad \text{MRR} = \frac{1}{N}\sum_i \frac{1}{r_i}.$$

Un indicatore separato, la citazione, registra se il riferimento compare anche nella risposta finale.

Per le acquisizioni si misura il richiamo, cioè la frazione dei *core_ids* attesi effettivamente recuperati, e la quota dei prodotti recuperati che cade sul luogo richiesto. I tre casi STAC con riferimento vuoto entrano nella classificazione degli esiti e sono esclusi dalle metriche di recupero.

8.7 Confronti e ripetizioni

8.7.1 Riproducibilità e stabilità

L'ordine dei casi dipende da un seed. Con una sola ripetizione il Judge opera a temperatura 0; negli studi di stabilità, con più ripetizioni, usa temperatura 0,3 e aggrega i voti con la mediana. Ogni run registra protocollo, impronta del dataset, modello Judge, temperatura, seed e numero di ripetizioni.

8.7.2 Confronti appaiati fra run

Due run possono essere confrontate caso per caso quando condividono protocollo, Judge e almeno 20 coppie valutabili per categoria. Il protocollo prevede il test di Wilcoxon sui voti e quello di McNemar sugli esiti, con correzione di Holm per i confronti multipli [35, 19, 37]. Prima di attribuire una differenza a una modifica del sistema, è buona pratica confrontare due run della stessa versione (confronto A/A).

8.8 Risultati

8.8.1 Riepilogo del protocollo 6.0

Sul dataset da 105 casi, la run `eval-run-ca004ba5` (protocollo 6.0, agente `qwen3:8b`, Judge `gemma3:27b`, temperatura zero, seed 40) ha prodotto i conteggi della Tabella 8.2. Il bootstrap usa 2.000 ricampionamenti delle famiglie [28].

La differenza fra i due tassi del grafo è dovuta interamente ai due casi in `review`.

Una seconda run completa con protocollo 6.0 sullo stesso dataset, con seed 42 (`721bec34`), ottiene 61,2% sul grafo e 98,1% sulle acquisizioni [28]. I risultati delle due run sono vicini e indicano una variabilità dell'ordine di pochi punti percentuali.

Categoria	Pass (primario)	Pass (rigoroso)	Wilson 95%	Bootstrap 95%
Grafo	31/49 (63,3%)	31/51 (60,8%)	49,3–75,3%	43,8–81,8%
Acquisizioni	54/54 (100%)	54/54 (100%)	93,4–100%	100–100%

Tabella 8.2: Conteggi della run `eval-run-ca004ba5`. Il tasso primario esclude i due casi `review` del grafo, il rigoroso li include. Per le acquisizioni il bootstrap collassa a 100% perché il campione contiene solo successi.

I 18 fallimenti del grafo si distribuiscono così: otto con riferimento assente dalla lista, quattro con errore di percorso e mancato recupero, quattro con riferimento citato ma voti insufficienti e due con lista vuota. I due casi `review` sono conteggiati a parte. Sui 49 casi valutabili, hit e citazione raggiungono il 71,4%, hit@1 il 18,4% e MRR 0,295. Per le acquisizioni, esclusi i tre casi con riferimento vuoto, il recupero completo dei *core_ids* è 94,1% su 51 casi e il richiamo medio è 97,4% [28].

Le cifre indicano dove intervenire. Sul grafo il riferimento è spesso recuperato (hit 71,4%) ma raramente in prima posizione (hit@1 18,4%): il punto debole è l'ordinamento dei risultati, più che il recupero. Sulle acquisizioni, invece, il 100% segnala un effetto soffitto: il benchmark corrente è troppo facile per distinguere una versione migliore da una peggiore e andrebbe esteso con casi più ambigui (luoghi omonimi, periodi relativi, combinazioni di filtri).

Precisione geografica e richiamo misurano proprietà diverse. STAC-017, per esempio, supera la valutazione recuperando tre dei sei prodotti di riferimento, tutti pertinenti al luogo: il controllo sul richiamo segnala un avviso `major`, che il protocollo riporta senza farne un motivo di fallimento. Un pass attesta quindi la correttezza della ricerca e della sintesi, mentre l'esaustività è descritta dagli indicatori di recupero.

8.8.2 Confronto storico con protocollo 4.0

La guida del 15 ottobre 2026 documenta run con protocollo 4.0 su un dataset da 115 casi, diverso dall'attuale. La più ampia, `7dc4eda3`, è parziale (102/115) e riporta 49/51 sul grafo e 32/51 sulle acquisizioni [29]. Poiché protocollo, regole critiche, composizione del dataset e completezza della run sono cambiati, queste cifre servono a collocare storicamente il lavoro e vanno lette separatamente dalla Tabella 8.2.

8.9 Interpretazione e minacce alla validità

Il contributo del protocollo consiste nel rendere ispezionabili domanda, evidenze, percorso, controlli e voti, distinguendo i fallimenti dell'agente dai problemi esterni e dai giudizi inutilizzabili. I risultati vanno letti alla luce di alcuni limiti, raccolti qui:

- *Judge.* Manca una validazione umana dei voti sul compito geospaziale, per cui eventuali falsi pass o fail restano possibili. Il Judge riceve inoltre l'esito dei controlli: il suo voto è informato dai fatti verificati e li integra, senza costituirne una conferma indipendente. Wilson e bootstrap descrivono l'incertezza del campione, ma presuppongono un Judge corretto.

- *Ground truth.* I riferimenti del grafo provengono dallo stesso catalogo NebulaGraph interrogato dall'agente: il benchmark misura la capacità di trovare risorse presenti nel catalogo, non la completezza del catalogo stesso. I controlli si basano su euristiche e le parafrasi della stessa famiglia sono dipendenti.

- *Copertura.* Le domande sono curate e le famiglie limitate; il set perturbato è escluso dai risultati di questo capitolo. Due sole run danno un'indicazione, e non una stima precisa, della variabilità.

- *Termini di confronto.* Manca un confronto con un agente singolo privo di workflow, che quantificherebbe il beneficio dell'architettura.

- *Interfaccia.* Il benchmark verifica la risposta dell'agente; la corrispondenza con tabella e mappa effettivamente visualizzate nel browser resta da controllare.

Per rafforzare le conclusioni servono una validazione umana di un campione di voti, confronti A/A, un termine di confronto a singolo agente, più scenari indipendenti e test dell'interfaccia del workspace.

Capitolo 9

Conclusioni e sviluppi futuri

Il lavoro ha realizzato *Copernicus Data Workspace*: un assistente per consultare fonti geospaziali tramite richieste in linguaggio naturale, esposto in una dashboard web autenticata con Keycloak. Il contributo è progettuale e implementativo: responsabilità specializzate e percorsi controllati. Sul piano progettuale sono stati ottenuti, in particolare:

- un workflow LangGraph con confine di dominio, parsing strutturato, percorso Discovery su NebulaGraph, percorso operativo multi-sorgente (web, GDACS, STAC live, CEMS) e percorsi combinati;
- integrazioni MCP per GDACS e CEMS, client STAC diretto per le acquisizioni, harvester e catalogo metadati Marine/STAC/Drought;
- memoria conversazionale Redis, artefatti dashboard verificabili (valori calcolati dal codice sui record recuperati), ripresa delle sessioni e interfaccia bilingue IT/EN;
- stack di osservabilità (Prometheus, Grafana, OpenTelemetry/Phoenix) per tempi, errori, nodi e dipendenze;
- un protocollo di valutazione del workflow (versione 6.0) con controlli deterministici, rubriche ordinali e stima dell'incertezza.

MCP delimita l'integrazione di GDACS e CEMS; CDSE STAC live resta un percorso diretto. Discovery e STAC live svolgono compiti distinti: il primo sceglie risorse dal catalogo a grafo; il secondo cerca scene sul catalogo remoto.

Risultati della valutazione. Sul dataset descritto da 105 casi, il riepilogo della run di riferimento del protocollo 6.0 (Judge `gemma3:27b`, temperatura 0) riporta sul grafo 31/49 (63,3% primario) e 31/51 (60,8% rigoroso), e sulle acquisizioni 54/54 (100%). I conteggi e il ricalcolo degli intervalli sono documentati nel rapporto di riscontro tecnico del 5 novembre 2026 [28]. Il Capitolo 8 ne esplicita criteri e denominatori. Il benchmark copre questi due percorsi; GDACS, CEMS, follow-up e ripresa sessione sono oggetto di altre verifiche.

Una seconda run con seed diverso ottiene valori vicini (61,2% e 98,1%). Il recupero sul grafo è buono, ma il riferimento è raramente al primo posto; sulle acquisizioni il benchmark è ormai troppo facile per distinguere miglioramenti.

Limiti e sviluppi. Le priorità sono il ranking della Discovery, un benchmark delle acquisizioni più difficile, la protezione delle API del backend con verifica del token e del proprietario degli artefatti, una baseline controllata sul contributo multiagente, la validazione umana del Judge e la verifica nel browser di risposta e artefatto. Sul piano operativo vanno ampliati telemetria, freschezza del catalogo e recupero dai guasti. L'obiettivo è mantenere ogni affermazione proporzionata all'evidenza disponibile.

Bibliografia

[1] Anthropic. Introducing the Model Context Protocol, 2024. Consultata il 5 novembre 2026. https://www.anthropic.com/news/model-context-protocol.

[2] Arize AI. Phoenix: Setup Tracing. Documentazione ufficiale consultata il 5 novembre 2026. https://arize.com/docs/phoenix/tracing/how-to-tracing/setup-tracing.

[3] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems*, 2020. https://arxiv.org/abs/2005.14165.

[4] Copernicus Data Space Ecosystem. APIs of the Copernicus Data Space Ecosystem. Documentazione ufficiale consultata il 5 novembre 2026. https://dataspace.copernicus.eu/analyse/apis.

[5] Copernicus Data Space Ecosystem. STAC Product Catalogue. Documentazione ufficiale consultata il 5 novembre 2026. https://documentation.dataspace.copernicus.eu/APIs/STAC.html.

[6] Copernicus Emergency Management Service. About Copernicus EMS. Pagina ufficiale consultata il 5 novembre 2026. https://emergency.copernicus.eu/about/.

[7] Copernicus Emergency Management Service. Rapid Mapping: Product Overview, 2025. Pagina ufficiale aggiornata il 31 gennaio 2025 e consultata il 5 novembre 2026. https://mapping.emergency.copernicus.eu/about/rapid-mapping-manual/product-overview/.

[8] Copernicus Emergency Management Service. Risk and Recovery Mapping: Service Overview, 2025. Pagina ufficiale aggiornata il 29 agosto 2025 e consultata il 5 novembre 2026. https://mapping.emergency.copernicus.eu/about/risk-and-recovery-manual/service-overview/.

[9] Docker. Docker Compose Documentation. Documentazione ufficiale consultata il 5 novembre 2026. https://docs.docker.com/compose/.

[10] Global Disaster Alert and Coordination System. GDACS Feed Reference. Pagina ufficiale consultata il 5 novembre 2026. https://www.gdacs.org/feed_reference.aspx.

[11] Global Disaster Alert and Coordination System. GDACS Knowledge: Scientific Overview. Pagina ufficiale consultata il 5 novembre 2026. https://data.gdacs.org/Knowledge/overview.aspx.

[12] Global Disaster Alert and Coordination System. GDACS Overview. Pagina ufficiale consultata il 5 novembre 2026. https://www.gdacs.org/About/overview.aspx.

[13] Grafana Labs. Provision Grafana. Documentazione ufficiale consultata il 5 novembre 2026. https://grafana.com/docs/grafana/latest/administration/provisioning/.

[14] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. ImageNet Classification with Deep Convolutional Neural Networks. In *Advances in Neural Information Processing Systems*, 2012. https://proceedings.neurips.cc/paper/4824-imagenet-classification-with-deep-convolutional-neural-networks.

[15] LangChain. LangChain Documentation. Documentazione ufficiale consultata il 5 novembre 2026. https://docs.langchain.com/oss/python/langchain/overview.

[16] LangChain. LangGraph Documentation. Documentazione ufficiale consultata il 5 novembre 2026. https://docs.langchain.com/oss/python/langgraph/overview.

[17] LangChain. LangGraph: Persistence. Documentazione ufficiale consultata il 5 novembre 2026. https://docs.langchain.com/oss/python/langgraph/persistence.

[18] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep Learning. *Nature*, 521:436–444, 2015. https://www.nature.com/articles/nature14539.

[19] Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947. https://doi.org/10.1007/BF02295996.

[20] Microsoft. From Questions to Discoveries: NASA's New Earth Copilot Brings Microsoft AI Capabilities to Democratize Access to Complex Data, 2024. Annuncio del 14 novembre 2024, consultato il 5 novembre 2026. https://blogs.microsoft.com/blog/2024/11/14/from-questions-to-discoveries-nasas-new-earth-copilot-brings-microsoft-ai-capabilities-to-democratize-access-to-complex-data/.

[21] Model Context Protocol. Model Context Protocol Specification, 2025. Versione 2025-06-18, consultata il 5 novembre 2026. https://modelcontextprotocol.io/specification/2025-06-18.

[22] NebulaGraph. NebulaGraph Database Manual: Architecture Overview. Documentazione della versione 3.8.0, consultata il 5 novembre 2026. `https://docs.nebula-graph.io/3.8.0/1.introduction/3.nebula-graph-architecture/1.architecture-overview/`.

[23] Ollama. API Usage: Performance and Model Usage Metrics. Documentazione ufficiale consultata il 5 novembre 2026. `https://docs.ollama.com/api/usage`.

[24] Ollama. Ollama Documentation. Documentazione ufficiale consultata il 5 novembre 2026. `https://docs.ollama.com/`.

[25] Open WebUI. Open WebUI Documentation. Documentazione ufficiale consultata il 5 novembre 2026. `https://docs.openwebui.com/`.

[26] OpenAI. Function Calling. Documentazione ufficiale consultata il 5 novembre 2026. `https://developers.openai.com/api/docs/guides/function-calling`.

[27] OpenTelemetry Authors. Traces. Documentazione ufficiale consultata il 5 novembre 2026. `https://opentelemetry.io/docs/concepts/signals/traces/`.

[28] Alessio Pecilli. Risposte alle verifiche: codice e tesi, 2026. Rapporto tecnico del 5 novembre 2026, allegato ai materiali del progetto come `RISPOSTE_VERIFICHE.md`. Riscontro del prototipo, del dataset e della run `eval-run-ca004ba5`; artefatti originali esterni al repository della tesi.

[29] Progetto PecilliSpace. Guida completa al progetto: GraphDB, LLM Judge e observability, 2026. Documento tecnico interno, ricognizione del 15 ottobre 2026. File `guida-graphdb-judge-observability.md`, fornito con i sorgenti della tesi. Contiene descrizione dell'implementazione, risultati salvati e verifiche documentali; i risultati citati non sono nuove esecuzioni.

[30] Prometheus Authors. Histograms and Summaries. Documentazione ufficiale consultata il 5 novembre 2026. `https://prometheus.io/docs/practices/histograms/`.

[31] Sebastián Ramírez and contributors. FastAPI Documentation. Documentazione ufficiale consultata il 5 novembre 2026. `https://fastapi.tiangolo.com/`.

[32] Redis. LangGraph Redis: Checkpoint Savers and Stores. Repository ufficiale consultato il 5 novembre 2026. `https://github.com/redis-developer/langgraph-redis`.

[33] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 4 edition, 2020. Capitolo 2, Intelligent Agents. Sito ufficiale della quarta edizione: `https://aima.cs.berkeley.edu/`.

[34] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems*, 2023. `https://proceedings.neurips.cc/paper_files/paper/2023/hash/d842425e4bf79ba039352da0f658a906-Abstract-Conference.html`.

[35] SciPy Community. Wilcoxon Signed-Rank Test. Documentazione ufficiale consultata il 5 novembre 2026. https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.wilcoxon.html.

[36] SpatioTemporal Asset Catalog. STAC Specification. Versione 1.1.0, consultata il 5 novembre 2026. https://github.com/radiantearth/stac-spec/tree/v1.1.0.

[37] statsmodels Developers. Multiple Testing: multipletests. Metodo Holm, documentazione ufficiale consultata il 5 novembre 2026. https://www.statsmodels.org/stable/generated/statsmodels.stats.multitest.multipletests.html.

[38] statsmodels Developers. Proportion Confidence Intervals: proportion_confint. Documentazione ufficiale consultata il 5 novembre 2026. https://www.statsmodels.org/stable/generated/statsmodels.stats.proportion.proportion_confint.html.

[39] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention Is All You Need. In *Advances in Neural Information Processing Systems*, 2017. https://arxiv.org/abs/1706.03762.

[40] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations*, 2023. https://arxiv.org/abs/2210.03629.

[41] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems*, volume 36, 2023. https://arxiv.org/abs/2306.05685.